Czech Technical University in Prague
Faculty of Electrical Engineering
Department of Computer Science and Engineering

Technical Report DCSE-TR-2008-0x

Continued Logarithms

Tomáš Brabec

September 2008
(September 29, 2008)

Contents

1 Introduction **1**

2 Continued Logarithm Representation **2**
2.1 Informal Introduction 2
2.2 Formal Definition 3
2.3 LFTs and Linear Algebra 6
2.4 Fundamental Properties 7

3 Continued Logarithm Arithmetic **14**
3.1 Basic Principles 15
3.2 Binary Operations 18
3.3 Further Details 22

4 Rational Extension **24**
4.1 Binary Extension 24
4.2 Canonical Extension 27
4.3 Revision of Arithmetic 28
4.4 Computability Issues 30

5 Real Extension **30**
5.1 Canonical Extension 30
5.2 Binary Extension 33
5.3 Revision of Arithmetic 37
5.4 Computability Issues 38
5.5 Speculative Redundancy 39

6 Conclusion **39**

7 Bibliography **40**

Continued Logarithms[1]

Tomáš Brabec
brabect1@fel.cvut.cz
Department of Computer Science and Engineering
Faculty of Electrical Engineering
Czech Technical University in Prague
Karlovo nám. 13, 121 35 Prague 2, CZ

Abstract: This manuscript covers the topic of *continued logarithms* as a potential representation for computable computer arithmetic. This kind of arithmetic is in practice known as *exact (real) arithmetic*. Continued logarithms were introduced by Gosper as an alternative to continued fractions, which has better properties for implementation in computers. Despite being proposed decades ago, their existence was almost neglected in the mainstream of the field. We revise here the continued logarithm representation and its arithmetic, formulate the basic principles and properties that make the representation interesting, and extend these principles to the point where the computable real arithmetic is established. Our aim is to summarize the basic knowledge and make it a common ground for other works dealing with some particular areas of continued logarithms.
WARNING: This text is a draft and is currently being reviewed. It may still contain some mistakes or lack soundness. There are also topics that are planned for future versions of this document.

Keywords computer arithmetic, exact real arithmetic, continued fractions, continued logarithms

1 Introduction

We provide here a detailed overview of continued logarithm representation, which was proposed in [Gosper, 1978]. Although Gosper's manuscript is well-known, the proposal of continued logarithms did not achieve much attention. We found this representation interesting, though, and explored its details, especially in connection with hardware implementation and application to exact arithmetic.

Gosper's original notes provide enough insight to understand the principles of continued logarithms and recognize their advantage and possible application to exact arithmetic. However, these notes are rather informal and sometimes very sketchy, thus easy to overlook or their importance to ignore. We aim here to formalize and extend Gosper's ideas, and summarize experience we gained while exploring the potential of continued logarithms. This document should form a basis that future publications may built on.

Continued logarithms themselves are introduced in Section 2. In the whole text, we work with two forms of the representation, the canonical and the binary form in particular. Both these forms are formally defined and their properties developed. Most of the statements in this section follow those of continued fractions [Khinchin, 1964] and so do their proofs.

In Section 3 we revise principles of arithmetic. They basically follow the well-known principles of continued fraction arithmetic as introduced and propagated by [Gosper, 1972]. Actually, the

[1]This manuscript evolved from a draft prepared in September 2006 and referred to in [Brabec, 2006].

similarity between continued logarithms and (regular) continued fractions goes along with us trough the whole document. No prior knowledge of continued fractions is necessary, however, though welcome.

Continued logarithms are in their simplest form limited to represent real numbers above 1. We address this limitation in Section 4 and show several possible solutions, discussing their pros and cons. We finally show that while the representation can represent any real number, it still suffers from some computability issues.

The computability is solved in Section 5, wherein we introduce a redundant variant of continued logarithms. We heavily rely on results of [Brabec, 2008], which proved admissibility of such an approach for canonical form of continued logarithms. Here we extend these results to the binary form as well.

In the final section, we provide short summary and propose further developments of this document. We omit here a conventional section on related work—interested readers may refer to e.g. [Gowland & Lester, 2000].

2 Continued Logarithm Representation

2.1 Informal Introduction

The notion of continued logarithm (CL) reflects its main characteristics—that is, continued fractions nature and logarithmic representation. Combining these two principles led to a number representation that, according to [Gosper, 1978], *"has several advantages over regular continued fractions, especially for computational hardware."* We will discuss properties of continued logarithms later (see Section 2.3), but let us just note that a fundamental one is a restricted alphabet.

Continued logarithms need theoretically only two bits of information (0 and 1) with their meaning going as follows [Gosper, 1978]: *"A 1 means 'I was at least 2, so I halved myself'. A 0 means 'I was between 1 and 2, so I subtracted 1 and reciprocated myself (so now I am > 1)'."* Illustrated by Example 1, this understanding also provides a procedure that finds such a representation.

Example 1. *Let us find a continued logarithm representation of a number $x = \frac{7}{2}$. Following the meaning of individual bits, we let $x_0 = x$ and iterate the formula below, obtaining bits b_i from left to right, until $i = n$ such that $x_n = 1$. In that case, we emit the last '0' and terminate the procedure.*

$$x_i \quad \rightarrow \quad \begin{array}{lllll} 2 \leq x_i & \Rightarrow & b_i = 1 & \rightarrow & x_{i+1} = \frac{x_i}{2} \\ 1 \leq x_i < 2 & \Rightarrow & b_i = 0 & \rightarrow & x_{i+1} = \frac{1}{x_i - 1} \end{array}$$

Thus, starting from $x_0 = \frac{7}{2}$ we obtain $x = (1001010)_{BCL}$:

$$\frac{7}{2} \xrightarrow{\text{'1'}} \frac{7}{4} \xrightarrow{\text{'0'}} \frac{4}{3} \xrightarrow{\text{'0'}} 3 \xrightarrow{\text{'1'}} \frac{3}{2} \xrightarrow{\text{'0'}} 2 \xrightarrow{\text{'1'}} 1 \xrightarrow{\text{'0'}} \infty\,.$$

□

The string representing a continued logarithm can be viewed as bursts of '1's delimited by a '0'. Considering such bursts we may write continued logarithms in an alternative form as $k_0 0 k_1 0 \ldots 0 k_n 0$ or just $k_0, k_1, \ldots, k_n$, where k_i is the length of the i-th burst. For instance, Gosper's example [Gosper, 1978], $100/2.54 = (1111101101110101110001101010 10)_{\mathrm{BCL}}$, can be expressed as $5, 2, 3, 1, 3, 0, 0, 2, 1, 1, 1$. Although this is just a variation with a string representation, such consideration establishes a connection between continued logarithms and continued fractions (see next section).

2.2 Formal Definition

A general continued fraction structure of a real number x (or just a *continued fraction*) has the following form:

$$x = a_0 + \cfrac{b_0}{a_1 + \cfrac{b_1}{a_2 + \cfrac{b_2}{\ddots + \cfrac{b_{i-1}}{a_i + \dots}}}}. \tag{2.1}$$

We call the symbols a_i and b_i *i-th level elements* of the continued fraction and they are usually constrained specifically for every class of continued fractions. Herein, we will consider two classes: *simple* (or *regular*) *continued fractions* and continued logarithms. In both cases, all elements are assumed to be integers. In particular, for simple continued fractions, $a_i \in \mathbb{Z}$ and $b_i = 1$, while for continued logarithms $a_i = b_i = 2^{k_i}$, $k_i \in \mathbb{N} \cup \{0\}$.

Due to their nature, continued fractions may be written recursively as

$$x_i = a_i + \frac{b_i}{x_{i+1}}, \quad i = 0, 1, \dots \tag{2.2}$$

Starting from $x_0 = x$, the recursion builds iteratively the continued fraction structure:

$$x = x_0 = a_0 + \frac{b_0}{x_1} = a_0 + \cfrac{b_0}{a_1 + \cfrac{b_1}{x_2}} = \dots = a_0 + \cfrac{b_0}{a_1 + \cfrac{b_1}{\ddots + \cfrac{b_{i-1}}{x_i}}} = \dots$$

Symbols x_i are called *continuations* as each of them represents an i-th yet folded part of the continued fraction. On the other hand, the already unfolded part of length $i + 1$ is called *i-th segment*. Thus for $i > 0$, the i-th segment together with the i-th continuation fully specify a continued fraction. We may also sporadically use the notion of *i-th remainder*[2] to denote $\frac{b_i}{x_{i+1}}$.

On the basis of the introduced notation, we may formally define representation by continued logarithms:

Definition 2. Canonical continued logarithm *representation (CCL) of a real number $x \geq 1$ is a sequence $\{k_i\}$, $k_i \in \mathbb{N} \cup \{0\}$, $i = 0, 1, \dots$, such that there is a sequence $\{x_i\}$, $x_i \in \mathbb{R} \setminus \{0\}$ and $x_0 = x$, satisfying*

$$2^{k_i} \leq x_i = 2^{k_i}\left(1 + \frac{1}{x_{i+1}}\right) < 2^{k_i+1}. \tag{2.3}$$

We see from $x_i = 2^{k_i} + \frac{2^{k_i}}{x_{i+1}}$ that $a_i = b_i$ as we mentioned earlier. Furthermore, the constraint $2^{k_i} \leq x_i < 2^{k_i+1}$ determines the choice of k_i as the closest power of 2 lower or equal to x_i, or equally

$$k_i = \lfloor \log_2 x_i \rfloor.$$

Since k_i's fully specify the value of a continued logarithm, we write

$$x = (k_0, k_1, \dots, k_i, \dots)_{\mathrm{CCL}}$$

[2]Beware of [Khinchin, 1964] using "remainder" to denote "continuation" in our terminology.

instead of the complete fractional form

$$x = 2^{k_0} + \cfrac{2^{k_0}}{2^{k_1} + \cfrac{2^{k_1}}{\ddots + \cfrac{2^{k_{i-1}}}{2^{k_i} + \dots}}}.$$

We call either form an *expansion*—a *continued logarithm expansion* in particular. In general, we may write any continued fraction given by Eq. (2.1) in an abbreviated form of e.g. $(a_0, b_0/a_1, b_1/\dots)$. However, thanks to constraints imposed on elements of certain classes, we may simplify their expansions accordingly. Thus e.g. we write continued logarithms as above, or regular continued fractions as $(a_0/a_1/\dots)$.

Example 3. *Let us find the CCL representation of* $x = \sqrt{10}$. *We start from* $x_0 = x$, *and determine its* $\log_2$ *magnitude* k_0. *From equality in (2.3) and values of* x_0 *and* k_0, *we determine* x_1 *as the next continuation and repeat the process:*

$$x_i \quad \to \quad k_i = \lfloor \log_2 x_i \rfloor \quad \to \quad x_{i+1} = \frac{2^{k_i}}{x_i - 2^{k_i}}. \tag{2.4}$$

First few results for the given x *are listed in Tab. 2.1, the first two of which are for illustration computed below.*

$$x_0 = \sqrt{10} = 3.16\dots \quad \to \quad k_0 = \lfloor 1.66\dots \rfloor = 1 \quad \to \quad x_1 = \frac{2}{\sqrt{10}-2},$$
$$x_1 = \frac{2}{\sqrt{10}-2} = 1.72\dots \quad \to \quad k_1 = \lfloor 0.78\dots \rfloor = 0 \quad \to \quad x_2 = \frac{\sqrt{10}-2}{4-\sqrt{10}}.$$

□

Table 2.1: CCL representation of $x = \sqrt{10}$.

i	0	1	2	3	4	5	6	...
x_i	3.16...	1.72...	1.38...	2.58...	3.44...	1.38...	2.58...	...
k_i	1	0	0	1	1	0	1	...

Definition 4. Binary continued logarithm *representation (BCL) of a real number* $x \geq 1$ *is a sequence of* bits $\{b_i\}$, $b_i \in \{0,1\}$, $i = 0, 1, \dots$, *such that there is a sequence* $\{x_i\}$, $x_i \in \mathbb{R} \setminus \{0\}$ *and* $x_0 = x$, *satisfying*

$$1 \leq x_{i+1} = \begin{cases} x_i/2, & \textit{for } x_i \geq 2 \\ 1/(x_i - 1), & \textit{otherwise}, \end{cases} \tag{2.5}$$

and

$$b_i = \begin{cases} 1, & \textit{for } x_i \geq 2 \\ 0, & \textit{otherwise}. \end{cases}$$

This definition of binary continued logarithms corresponds exactly to that of Gosper given in Section 2.1. We may invert the relation (2.5) and write

$$1 \leq x_i = \begin{cases} 2x_{i+1} & \text{for } x_i \geq 2, \\ 1 + \frac{1}{x_{i+1}} & \text{otherwise}. \end{cases} \tag{2.6}$$

Let us now have BCL representation of some number and take n such that $b_n = 0$ and thus $x_{n+1} = 1/(x_n - 1)$. If $k = \lfloor \log_2 x_{n+1} \rfloor$, then $k \geq 0$ and

$$\begin{aligned} x_{n+1} &= \begin{cases} 2x_{n+2} = \ldots = 2^k \left(1 + \frac{1}{x_{n+2+k}}\right) & \text{for } k > 0, \\ 1 + \frac{1}{x_{n+2}} & \text{for } k = 0, \end{cases} \\ &= 2^k \left(1 + \frac{1}{x_{n+2+k}}\right). \end{aligned} \tag{2.7}$$

This result establishes a one-to-one correspondence between BCL and CCL representations, formally expressed by the following theorem. It also shows that CCL is actually the "bursted" notation used in Section 2.1.

Theorem 5 (Equivalence of BCL and CCL)**.** *Let $\{b_i\}$ be a BCL representation of a real number $x \geq 1$. We define a sequence $\{n_j\}$, $j = 1, 2, \ldots$, such that:*

1. *$n_1 < n_2 < \ldots$, and*
2. *$\forall j : \; b_{n_j} = 0$, and*
3. *$\forall j$: there is no m, $n_j < m < n_{j+1}$, such that $b_m = 0$.*

In other words, $\{n_j\}$ is the ordered sequence of indices of all zero bits within $\{b_i\}$. Then for a CCL representation $\{k_l\}$ of x it holds:

$$k_0 = n_1 \quad \text{and} \quad k_j = n_{j+1} - n_j - 1, \; j = 1, 2, \ldots$$

Proof. We will prove the equivalence by showing equivalence of the determined continuations[3] in both representations. Let $\{x_i\}$ be a sequence of BCL continuations defined recursively by Eq. (2.5). Similarly, let $\{y_l\}$ be a sequence of CCL continuations defined recursively by Eq. (2.3). We will then show that for $j = 1, 2, \ldots$

$$x_{n_j+1} = y_j.$$

By induction:

1. $j = 1$:
   From Eq. (2.3) we get

   $$x = y_0 = 2^{k_0} \left(1 + \frac{1}{y_1}\right) \qquad \Rightarrow \qquad y_1 = \frac{1}{1 - x2^{-k_0}}.$$

   Similarly from Eq. (2.5)

   $$x_{n_1} = \begin{cases} x & \text{if } n_1 = 0, \\ x_{n_1-1}2^{-1} = \ldots = x2^{-n_1} & \text{otherwise,} \end{cases} \qquad \Rightarrow \qquad x_{n_1+1} = \frac{1}{1 - x_{n_1}} = \frac{1}{1 - x2^{-n_1}}.$$

   Since $k_0 = n_1$, then also $y_1 = x_{n_1+1}$.

[3] Although BCLs are not really continued fractions in the sense of Eq. (2.1), their correspondence with CCL justifies the use of continued fractions terminology.

2. $j = m$:
   Suppose that the induction assumption holds for $m-1$, i.e. $y_{m-1} = x_{n_{m-1}+1}$.
   For CCL it is

$$y_{m-1} = 2^{k_{m-1}}\left(1 + \frac{1}{y_m}\right) \qquad \Rightarrow \qquad y_m = \frac{1}{1 - y_{m-1}2^{-k_{m-1}}}.$$

   For BCL it is by Eq. (2.7)

$$x_{n_{m-1}+1} = 2^{n_m - n_{m-1} - 1}\left(1 + \frac{1}{x_{n_m+1}}\right) \qquad \Rightarrow \qquad x_{n_m+1} = \frac{1}{1 - x_{n_{m-1}+1}2^{-(n_m - n_{m-1} - 1)}}.$$

   Since $y_{m-1} = x_{n_{m-1}+1}$ and $k_{m-1} = n_m - n_{m-1} - 1$, then also $y_m = x_{n_m+1}$.

□

2.3 LFTs and Linear Algebra

Linear Fractional Transformation (LFT) is a transformation of the form

$$\phi(z) = \frac{az + b}{cz + d}\ ,$$

where $a, b, c, d \in \mathbb{C}$ and $ad - bc \neq 0$. The transformation can be extended to the entire extended complex plane $\mathbb{C}^* = \mathbb{C} \cup \{\infty\}$ by defining $\phi(-\frac{d}{c}) = \infty$ and $\phi(\infty) = \frac{a}{c}$. The case of $ad - bc = 0$ is not a particularly interesting as $\phi(z) = \frac{a}{c} = \text{const}$.

LFTs have a number of interesting properties. For us, the most important here is that the set of all LFTs forms a group under composition and that every LFT $\phi(z) = (az + b)/(cz + d)$ has an inverse

$$\phi^{-1}(z) = \frac{dz - b}{-cz + a}\ ,$$

such that $\phi^{-1} \circ \phi(z) = \phi \circ \phi^{-1}(z) = z$.

For purposes of this text, use of the complete $\mathbb{C}^*$ is not necessary and we restrict ourselves to a sub-group such that coefficients are integers, $a, b, c, d \in \mathbb{Z}$, and z is an extended real variable[4], $z \in \mathbb{R}^*$. This sub-group retains the above properties.

Principles of LFTs may be extended to multiple variables, thus introducing multi-linear fractional transformations, i.e. fractional transformations that are linear in all variables. Of particular interest here is a *bilinear fractional transformation* (BLFT)

$$\phi(x, y) = \frac{axy + by + cx + d}{exy + fy + gx + h}\ .$$

Again, we assume $a, b, c, d, e, f, g, h \in \mathbb{Z}$ and $x, y \in \mathbb{R}^*$. Even the multi-linear transformations are closed under composition with LFTs.

LFTs are closely related to matrices by making elements of a matrix $A = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ represent coefficients of a LFT $\phi_A(x) = \frac{ax+b}{cx+d}$ and writing $A \cong \phi_A$. If we extend $\cong$ to vectors so that $\left(\begin{smallmatrix} a \\ b \end{smallmatrix}\right) \cong \frac{a}{b}$ and consider $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} x \\ 1 \end{smallmatrix}\right) = \left(\begin{smallmatrix} ax+b \\ cx+d \end{smallmatrix}\right)$, we get the following result

$$\phi_A(x) \cong A \cdot \left(\begin{smallmatrix} x \\ 1 \end{smallmatrix}\right).$$

[4]We use $\mathbb{R}^*$ to denote an extended real line $\mathbb{R} \cup \{\infty\}$ as a one-point compactification [Heckmann, 1999] of a real line $\mathbb{R}$. This includes conventional extension of arithmetic operations such as $-\infty = \infty$, $\frac{x}{0} = \infty$ if $x \neq 0$, etc. For completeness, one should also consider undefined outcome of operations such as $\frac{\infty}{\infty}$ or $\infty - \infty$, but we omit this aspect here for simplicity.

There are further similarities between matrices and LFTs. Matrices form a group closed under multiplication and it is easy to verify for matrices $A \cong \phi_A$ and $B \cong \phi_B$ that

$$A \cdot B \cong \phi_A \circ \phi_B.$$

That is, matrix multiplication is equivalent to LFT composition. As with LFTs, it makes sense to consider non-singular matrices such that $ad - bc = \det A \neq 0$. A small difference, making the mapping $\cong$: $A \mapsto \phi_A$ surjective, is seen in that $kA \cong \phi_A$ for a constant $k \in \mathbb{Q} \setminus \{0\}$. That is, any common factor of a matrix may be canceled without affecting the related LFT. For this reason, it makes sense to consider a matrix *pseudo-inverse* A^* [Heckmann, 1999] instead of the real inverse A^{-1}. The pseudo-matrix is defined as

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^* = \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

and it is easy to verify that

$$A \cdot A^* = A^* \cdot A = \det A \cdot E \ ,$$

where $E = \left(\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right)$. Thus $\phi_A^{-1} \cong A^*$.

Thinking of LFTs as matrices has advantages especially in replacing compositions with matrix multiplication, where the latter is easier at least when computed by hand. Therefore, in the following text we will use matrices and LFTs interchangeably, preferring the one that will be more appropriate in a situation.

We may also extend $\cong$ mapping to BLFTs so that a two-by-four matrix represents coefficients of a BLFT, thus writing $\left(\begin{smallmatrix} a & b & c & d \\ e & f & g & h \end{smallmatrix}\right) \cong \phi$ if $\phi(x, y) = \frac{axy+by+cx+d}{exy+fy+gx+h}$. Unfortunately, some of the linear algebra properties are lost—e.g. a matrix multiplication cannot replace LFT composition. However, using the two-by-four matrices still makes sense, at least as a compact form of a BLFT.

2.4 Fundamental Properties

Continued fractions, and especially the class of regular continued fractions, have some interesting properties. In this section, we will discuss these properties in context of continued logarithms, showing that they are competitive to regular continued fractions for purposes of number representation.

This essential result grants continued logarithms the potential for representing real numbers. By default, continued logarithms are limited to a subset of real numbers in interval $[1, \infty)$, but we will show later (Section 4) a way to overcome this limitation.

LFT Character

All continued fractions in general have a *LFT character*, meaning that they can be written as a composition of LFTs. This comes form Eq. (2.2) as if we define

$$\phi_i(x) = \frac{a_i x + b_i}{c_i x + d_i}$$

and let $c_i = 1$ and $d_i = 0$, we may write Eq. (2.2) as

$$x_i = \phi_i(x_{i+1}).$$

By following the recursion, we get

$$x_0 = \phi_0(x_1) = \phi_0 \circ \phi_1(x_2) = \phi_0 \circ \phi_1 \circ \ldots \circ \phi_i(x_{i+1}) = \ldots$$

Using the matrix notation, we may "encode" i-th elements into an *i-th digit matrix* $\left(\begin{smallmatrix} a_i & b_i \\ 1 & 0 \end{smallmatrix}\right)$ and write a continued fraction x as

$$x \cong \prod_i \begin{pmatrix} a_i & b_i \\ 1 & 0 \end{pmatrix} .$$

We will use this notation quite frequently as it makes it easier to express both the representation and transformations of digit matrices.

This general result may be specialized to continued logarithms by defining digit matrices K_{k_i} and B_{b_i}, for CCL and BCL respectively, as follows:

$$K_{k_i} = \begin{pmatrix} 2^{k_i} & 2^{k_i} \\ 1 & 0 \end{pmatrix} \qquad \text{and} \qquad B_{b_i} = \begin{cases} \left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right) & \text{for } b_i = 1, \\ \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right) & \text{for } b_i = 0. \end{cases}$$

We also reserve symbols κ_{k_i} and β_{b_i} for corresponding LFTs.

Finite and Infinite Fractions

So far we carefully avoided the question of length of a continued fraction expansions. In definitions and theorems, we behaved like if all expansions were infinite or non-terminating (i.e. of infinite length). *Infinite* is also how we call continued fractions with infinite expansions. However, we have seen in Example 1 that continued logarithms may also be of finite length, i.e. *finite* continued logarithms. Formally,

$$x = (a_0, b_0/a_1, \ldots, b_{n-1}/a_n) = a_0 + \cfrac{b_0}{a_1 + \cfrac{b_1}{\ddots + \cfrac{b_{n-1}}{a_n}}}$$

is a finite continued fraction of length $n+1$. We say that such a continued fraction (or its expansion) *terminates at a_n* (or *for n*).

We may equally write finite continued fractions as a finite product of their digit matrices. Unfortunately, such a product is a matrix too and thus a LFT. If all the elements a_i and b_i are integers (as we actually assume), then a finite continued fraction represents a rational number—not a LFT any more. This is where ∞ comes as a "termination mark" as seen from $a_n = \lim_{x\to\infty}(a_n + \frac{b_n}{x})$. That is, we can terminate a continued fraction at a_n by letting $x_{n+1} = \infty$. Since for a LFT $\phi(x) = \frac{ax+b}{cx+d}$ we defined $\phi(\infty) = \frac{a}{c}$, we thus get by equivalence $\left(\begin{smallmatrix} a \\ c \end{smallmatrix}\right) = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} \infty \\ 1 \end{smallmatrix}\right)$. Thus finally, the matrix notation of a finite continued fraction comes out as

$$x = (a_0, b_0/a_1, \ldots, b_{n-1}/a_n) \cong \prod_0^n \begin{pmatrix} a_i & b_i \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} \infty \\ 1 \end{pmatrix} .$$

Although we usually omit ∞ from an expansion, its understanding as a termination of a continued fraction is quite important. We will see in Section 3 that it is one of the keystones of continued fraction arithmetic.

As mentioned, finite continued fractions with integer elements represent rational numbers. This is also the case of continued logarithms and regular continued fractions as well. However, not much to say about infinite continued fractions in general. But we may be more specific for certain classes. E.g. regular continued fractions were shown [Khinchin, 1964] to have infinite expansions iff they represent irrational numbers. This is also the case of continued logarithms.

Theorem 6. *Let $x \geq 1$ be a real number. Then its continued logarithm (of CCL form) is finite if and only if x is rational, and infinite if and only if x is irrational.*

Proof. The proof relies on uniqueness of the continued logarithm representation, which is established later in this section (Theorem 7). For this moment, we just assume that for x there exists only a single CCL representation $(k_0, k_1, \ldots)_{\mathrm{CCL}}$.

There are two assertions in the theorem, each of which is a complementary to the other one thanks to the uniqueness. That is, CCL expansion of x is exclusively either finite or infinite, as it is x's rationality and irrationality. It thus suffices to prove one of the assertions and the other one will hold too. We thus show the assertion on finite expansions.

1. Finite expansion $\Rightarrow$ rational number:
   For finite expansions of length n we have

$$x = \prod_{i=0}^{n-1} \begin{pmatrix} 2^{k_i} & 2^{k_i} \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} \infty \\ 1 \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} \infty \\ 1 \end{pmatrix} = \frac{a}{c},$$

   where a, b, c, d are integers. Then x is a rational number.

2. Finite expansion $\Leftarrow$ rational number:
   By definition of continued logarithm we have

$$1 \le a_i \le x_i = a_i \left(1 + \frac{1}{x_{i+1}}\right), \qquad x_0 = x \text{ and } a_i = 2^{k_i} = 2^{\lfloor \log_2 x_i \rfloor} \text{ for } i = 1, 2, \ldots$$

   Let us choose n such that x_n is defined. Then x_n is either a power of 2, or not. In the former case, we get $x_n = a_n$ and let $x_{n+1} = \infty$, i.e. the continued logarithm terminates for n. We thus assume the latter case, where x_n is not a power of 2. Since x is rational, then every x_i will be rational too, and we let $x_n = p/q$ for some positive integers p, q. Then

$$1 > \frac{1}{x_{n+1}} = \frac{x_n - a_n}{a_n} = \frac{p - qa_n}{qa_n}.$$

   The equality $1 = \frac{1}{x_{n+1}}$ is not possible as we assume x_n not being a power of 2. Since $1/x_{n+1}$ is positive, we have $p > qa_n$ and it follows

$$x_{n+1} = \frac{qa_n}{p - qa_n} = \frac{p'}{q'} \qquad \text{and} \qquad 0 < p' < p,\ 0 < q' < p,$$

   i.e. x_{n+1} have strictly smaller (but positive) numerator than x_n has. Hence x must have a finite continued fraction as the numerators of x_i monotonously decrease toward 1 with increasing i, and thus there can be only a finite number of them.

□

Uniqueness

Uniqueness (or *non-redundancy*) of a representation defines that there is a one-to-one correspondence between a numerical value and its representation. From one point of view, uniqueness is an excellent property, thanks to which we can easily distinguish between two different values just by a difference in their representation. On the other hand, we will show later that some sort of redundancy is indispensable for real number computability (see Section 4.4).

There are two forms of continued logarithms—CCL and BCL, the first of which can be shown to be unique. By equivalence, the uniqueness must also hold for BCL too, but we will discuss below a certain aspect that affects this uniqueness.

Theorem 7 (Uniqueness)**.** *For every real number $x \ge 1$ there exists a unique continued logarithm (of CCL form) equal to x.*

Proof. Suppose that x is represented by two continued logarithms of expansions $(k'_0, k'_1, \ldots)_{\mathrm{CCL}}$ and $(k''_0, k''_1, \ldots)_{\mathrm{CCL}}$. We will show that it must then be $k'_i = k''_i$ for all i.

1. $i = 0$:
   The definition sets that $2^{k_0} \leq x = x_0 = 2^{k_0}\left(1 + \frac{1}{x_1}\right) < 2^{k_0+1}$, which implies that $k'_0 = \lfloor \log_2 x \rfloor = k_0$ and $k''_0 = \lfloor \log_2 x \rfloor = k_0$. Therefore $k'_0 = k''_0$.

2. $i = n + 1$:
   Suppose that the assumption $k'_i = k''_i$ holds for $i = 0, 1, \ldots, n$. Then

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \prod_{i=0}^{n} \begin{pmatrix} 2^{k'_i} & 2^{k'_i} \\ 1 & 0 \end{pmatrix} = \prod_{i=0}^{n} \begin{pmatrix} 2^{k''_i} & 2^{k''_i} \\ 1 & 0 \end{pmatrix} \quad \text{and} \quad x \cong A \cdot \begin{pmatrix} x'_{n+1} \\ 1 \end{pmatrix} \cong A \cdot \begin{pmatrix} x''_{n+1} \\ 1 \end{pmatrix},$$

   where $x'_{n+1} = (k'_{n+1}, k'_{n+2}, \ldots)_{\mathrm{CCL}}$ and $x''_{n+1} = (k''_{n+1}, k''_{n+2}, \ldots)_{\mathrm{CCL}}$ are $(n+1)$-th continuations of the first and second representation respectively. It follows that

$$x'_{n+1} = x''_{n+1} \cong A^* \cdot \begin{pmatrix} x \\ 1 \end{pmatrix} \cong \frac{dx - c}{-bx + a}.$$

   Since $k'_{n+1} = \lfloor \log_2 x'_{n+1} \rfloor$ and $k''_{n+1} = \lfloor \log_2 x''_{n+1} \rfloor$, it is $k'_{n+1} = k''_{n+1}$.

□

The slight difference between CCL and BCL is in the way of terminating the continued logarithm expansion. In the former case, CCL expansion terminates for such an n, for which $x_n = 2^{k_n}$ and x_{n+1} is considered infinite (and so is k_{n+1}). BCL equals CCL in that every k_i is represented as a burst of '1' bits of length k_i, followed by a '0' bit. Thus if we were to follow CCL's termination condition, then k_{n+1} would come out as an infinite stream of '1's. This indeed conforms to Eq. (2.5) because of $\frac{\infty}{2} = \infty$. This result is similar to the aspect "trailing zeros" known from conventional decimal representation.

There are reasons of practical use for avoiding the trailing '1's in BCL expansions. The basic reason is that infinite representation cannot be represented by finite computer resources. Aside of this fact, the numerical value is fully determined after processing the last 0 bit of its BCL representation. Processing any trailing '1's just unnecessarily increases the computation time. The aspect of trailing ones also affects our consideration of finite vs. infinite continued logarithm expansions—with trailing '1's, we have no finite BCL expansion and thus lose the simple tool for detecting rational numbers.

Therefore, hereinafter we agree to avoid "unnecessary" trailing '1's of a BCL representation whenever possible. By doing this, we immediately lose the uniqueness, at least in its pure form, because of the following equality:

$$(b_0 b_1 \ldots b_n)_{\mathrm{BCL}} = (b_0 b_1 \ldots b_n 1)_{\mathrm{BCL}} = (b_0 b_1 \ldots b_n 1 \ldots 1)_{\mathrm{BCL}} = (b_0 b_1 \ldots b_n 1 \ldots 1 \ldots)_{\mathrm{BCL}}.$$

We could have instead agreed to avoid "all" trailing '1's and thus retaining the uniqueness. However, we have reasons (evident in Section 5.4, Example 35), for which we allow to have some "necessary" number of trailing '1's appended to the representation. These reasons are related to the fact that the trailing '1's carry additional information on precision of the numerical value being represented. Consider e.g. decimal representations of 0.123 and 0.12300; while they both likely represent the same value, the former is precise up to $\pm 10^{-3}$, while the latter to $\pm 10^{-5}$. In fact, this gives BCL a slight advantage as for CCL there is no means to "encode" the partial information on k_{n+1} that would indicate $k_{n+1} > k$ for some k.

Finally, let us note that, despite of our agreement making BCL representation partially redundant, we recognize it here and in the following text as a *non-redundant*, because the notion of *redundancy* here, in this document, is mostly concerned with redundancy on purpose of computability.

Interval Arithmetic

Classes of continued fractions are distinguished by constraints of their elements. For continued logarithms, these constraints are implied by the recursive relation between continuations and by the bounded domain of each continuation. Take e.g. CCL defined by Eq. (2.3). An i-th continuation x_i is bounded to $[2^{k_i}, 2^{k_i+1})$, where $k_i = \lfloor \log_2 x_i \rfloor$. Bounded domain of x_i and the relation $\kappa_i(x) = 2^{k_i}(1 + \frac{1}{x})$, by which $x_i = \kappa_i(x_{i+1})$, imply together that $x_{i+1} \in (1, \infty]$. Due to equivalence of BCL and CCL, it is no wonder that Eq. (2.5) implies (almost) exactly the same constraint; i.e. $x_{i+1} \in [1, \infty]$.

What we would like to say is that each continued fraction class has its continuations naturally restricted somehow. E.g. for continued logarithms, we can rely on that every x_i will be in $[1, \infty]$ without actually knowing its value. It is like knowing that every digit d_i of a decimal expansion is between 0 and 9.

This gives us a chance to work with continued fractions like with interval arithmetic. Consider a LFT $\phi(x) = \frac{ax+b}{cx+d}$ over some domain $\mathbb{D}$, where $(x_l, x_h) \subseteq \mathbb{D} \subseteq [x_l, x_h]$ for some $x_l, x_h \in \mathbb{R}^*$. We will use the notation

$$\gamma(\mathbb{D}) = \{\gamma(x) \mid x \in \mathbb{D}\}$$

to define the range of a real function γ over $\mathbb{D}$. If $-\frac{d}{c} \in \mathbb{D}$, then $\phi(\mathbb{D}) = \mathbb{R}^*$, otherwise $(y_l, y_h) \subseteq \phi(\mathbb{D}) \subseteq [y_l, y_h]$, where $y_l = \phi(x_l)$ and $y_h = \phi(x_h)$ for increasing ϕ and vice versa for decreasing ϕ. The inclusion of y_l and y_h in $\phi(\mathbb{D})$ depends on inclusion of x_l and x_h in $\mathbb{D}$ and monotonicity orientation of ϕ.

Thus, for a n-th segment of a continued fraction, we may write

$$x \cong \prod_{i=0}^{n} \begin{pmatrix} a_i & b_i \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} x_{n+1} \\ 1 \end{pmatrix} \in \prod_{i=0}^{n} \begin{pmatrix} a_i & b_i \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} \mathbb{D} \\ 1 \end{pmatrix},$$

for a domain $\mathbb{D}$ implied by a particular continued fraction class. E.g. for continued logarithms $\mathbb{D} = (1, \infty]$.

Convergents and Approximation

A notion of convergent is tightly coupled with theory of approximation by continued fractions and to introduce it, we quote Khinchin's definition. Even though this definition primarily relates to regular continued fractions, it can be generalized to other classes as well.

> *"In the theory of continued fractions, an especially important role is played by the canonical representations of the segments of a given (finite or infinite) continued fraction $\alpha = [a_0; a_1, a_2, \ldots]$. We shall denote p_k/q_k the canonical representation of the segment $s_k = [a_0; a_1, a_2, \ldots, a_k]$ of the continued fraction, and we shall call it k-th order convergent (or approximant)."* [Khinchin, 1964]

The importance of convergents originates from the fact that the sequence of convergents $\{p_i/q_i\}$, $i = 0, 1, \ldots$, represents a Cauchy sequence of rational approximations to the represented value. The higher the order of a convergent, the better the approximation. In other words, if we take any finite *prefix* (i.e. segment) of a regular continued fraction expansion, this prefix defines a rational number approximating a value represented by the original expansion. If we later extend this prefix by further partial quotients, we will improve its proximity to the represented value. Therefore, regular continued fractions represent an implicitly *incremental number representation* and processing them "left-to-right" allows us to naturally compute with arbitrarily precise numerical values (see Section 3). Moreover, Khinchin showed

[Khinchin, 1964] that convergents of regular continued fractions are very good approximations, actually the best ones[5].

Definition 8. *An n-th order continued logarithm convergent of a real number x is a rational number equal to the value of n-th segment of x's CCL representation. Formally, if $\frac{p_n}{q_n}$ denotes the n-th order convergent, then*

$$\frac{p_n}{q_n} = (k_0, k_1, \ldots, k_n)_{CCL} \cong \prod_{i=0}^{n} \begin{pmatrix} 2^{k_i} & 2^{k_i} \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} \infty \\ 0 \end{pmatrix}.$$

The question is, however, if the "approximating" character of convergents of regular continued fractions preserves for convergents of other classes of continued fractions, continued logarithms in particular. It turns out that continued logarithms are well defined in this sense. That is, the sequences of their convergents converge to represented values, though the convergence speed is somewhat slower than for regular continued fractions. This can be illustrated by comparing results in Tab. 2.3 and 2.4.

The analytical formula for the value of an i-th order convergent of a canonical continued logarithm is defined as follows:

$$\frac{p_i}{q_i} = \frac{2^{k_i} p_{i-1} + 2^{k_{i-1}} p_{i-2}}{2^{k_i} q_{i-1} + 2^{k_{i-1}} q_{i-2}}, \quad i = 0, 1, \ldots, \tag{2.8}$$

where $p_{-1} = q_{-2} = 1$ and $p_{-2} = q_{-1} = 0$. This recursive formula can be derived either directly (by expanding segments of CCL representation), or using the following equivalence (provable by induction):

$$\begin{aligned} \begin{pmatrix} p_n & 2^{k_n} p_{n-1} \\ q_n & 2^{k_n} q_{n-1} \end{pmatrix} &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \cdot \prod_{i=0}^{n} \begin{pmatrix} 2^{k_i} & 2^{k_i} \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} p_{-1} & p_{-2} \\ q_{-1} & q_{-2} \end{pmatrix} \cdot \prod_{i=0}^{n} \begin{pmatrix} 2^{k_i} & 2^{k_i} \\ 1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} p_{n-1} & 2^{k_{n-1}} p_{n-2} \\ q_{n-1} & 2^{k_{n-1}} q_{n-2} \end{pmatrix} \cdot \begin{pmatrix} 2^{k_n} & 2^{k_n} \\ 1 & 0 \end{pmatrix}. \end{aligned}$$

When discussing uniqueness, we have seen that the binary form of continued logarithms (BCL) provides slightly more information than the canonical form (CCL). That is, k continuous 1 bits provide us with partial information that logarithmic magnitude is not less than k. We may define *pseudo-convergents*[6] as the value of segments of BCL expansions. Their values collide with those of the "canonical" convergents at every '0' bit of BCL expansion. However, due to the finer granularity, their associated matrices provide tighter bounds on the represented quantity (see Tab. 2.2).

Similarly to convergents of regular continued fractions, a number of properties hold also for convergents of continued logarithms. One of the most interesting is that subsequences composed of even order and odd order convregents respectively represent sequences of lower and upper bounds of the represented value. This means that a distance between two consecutive convergents (i.e. an absolute value of Eq. (2.9)) defines a maximum error that we commit by approximating a value by an i-th order convergent of its continued logarithm. This information may thus serve as an estimation of precision of a continued logarithm segment computed so far. Illustrative figures are given in Tab. 2.3.

[5] "The best" means that if a rational number p/q is a convergent of a simple continued fraction for a number x, then there is no other rational number p'/q', $q' < q$, such that $|q'x - p'| < |qx - p|$.

[6] *Pseudo* indicates that these values, despite of being approximants, do not conform to properties discussed for real convergents.

Table 2.2: Bounds on represented quantity implied by segments of CCL and BCL expansions of $x = 100/2.54$. Indices i and j correspond to CCL and BCL respectively.

i	$\frac{p_i}{q_i}$	i-th bound	j	$\frac{p_j}{q_j}$	j-th bound
-1	$\frac{1}{0}$	$[1,\infty]$	-1	$\frac{1}{0}$	$[1,\infty]$
			0	$\frac{2}{0}$	$[2,\infty]$
			1	$\frac{4}{0}$	$[4,\infty]$
			4	$\frac{32}{0}$	$[32,\infty]$
0	$\frac{32}{1}$	$[32,64)$	5	$\frac{32}{1}$	$[32,64)$
			6	$\frac{64}{2}$	$[32,48]$
			7	$\frac{128}{4}$	$[32,40]$
1	$\frac{160}{4}$	$(38.4,40]$	8	$\frac{160}{4}$	$[38.4,40)$
			9	$\frac{320}{8}$	$[39.1,40]$
...	...	...	...	...	...

Theorem 9. *For the difference of two consecutive continued logarithm convergents it is*

$$\frac{p_i}{q_i} - \frac{p_{i-1}}{q_{i-1}} = \frac{(-1)^i \prod_{n=0}^{i-1} 2^{k_n}}{q_i q_{i-1}}, \quad i = 1, 2, \ldots \tag{2.9}$$

Proof. From Eq. (2.8) we get

$$\begin{aligned}
\alpha_n &= q_{n-1}p_n - p_{n-1}q_n \\
&= q_{n-1}(2^{k_n}p_{n-1} + 2^{k_{n-1}}p_{n-2}) + p_{n-1}(2^{k_n}q_{n-1} + 2^{k_{n-1}}q_{n-2}) \\
&= -2^{k_{n-1}}(q_{n-2}p_{n-1} - p_{n-2}q_{n-1}) \\
&= -2^{k_{n-1}}\alpha_{n-1},
\end{aligned}$$

with $\alpha_0 = 1$. Unfolding the recursion we get the result $\alpha_n = (-1)^{n-1}\prod_{i=0}^{n-1} 2^{k_i}$. □

Corollary 10. *Continued logarithm convergents form a Cauchy sequence.*

Proof. Since for $n \geq 1$ it is $q_n = 2^{k_n}q_{n-1} + 2^{k_{n-1}}q_{n-2} \geq 2^{k_n}q_{n-1} + 1$, then $q_n \geq 1 + \sum_{i=0}^{n} 2^{\sum_{j=0}^{i} k_j} > \prod_{i=0}^{n} 2^{k_i}$. This is quite a rough lower bound and a better one could be found when considering $k_i = 0$ for all i; the sequence $\{q_j\}_{j=-2}^{n}$ then becomes a Fibonacci sequence and thus $q_n = F_{n+2}$, where F_i is an i-th term of Fibonacci sequence. Therefore

$$\frac{p_n}{q_n} - \frac{p_{n-1}}{q_{n-1}} = \frac{(-1)^n \prod_{i=0}^{n-1} 2^{k_i}}{q_n q_{n-1}} < \frac{(-1)^n}{q_{n-1}} \leq \frac{(-1)^n}{F_{n+1}},$$

which can be found arbitrarily small for some n. This result, together with $|\frac{p_n}{q_n} - \frac{p_{n-1}}{q_{n-1}}| < |\frac{p_{n-1}}{q_{n-1}} - \frac{p_{n-2}}{q_{n-2}}|$, completes the proof. □

Theorem 11. *Let us have a continued logarithm representation of a real number x. Then its even order convergents form an increasing and odd order convergents a decreasing sequence. Moreover, every odd order convergent is greater than any preceding even order convergent. Therefore*

$$\frac{p_0}{q_0} < \ldots < \frac{p_{2n}}{q_{2n}} < \ldots \leq x \leq \ldots < \frac{p_{2n+1}}{q_{2n+1}} < \ldots < \frac{p_1}{q_1}.$$

Proof. We first determine a difference between n-th and $(n-2)$-th order convergent.

$$\begin{aligned}\frac{p_n}{q_n} - \frac{p_{n-2}}{q_{n-2}} &= \frac{p_n}{q_n} - \frac{p_{n-1}}{q_{n-1}} + \frac{p_{n-1}}{q_{n-1}} - \frac{p_{n-2}}{q_{n-2}} = \frac{(-1)^n \prod_{i=0}^{n-1} 2^{k_i}}{q_n q_{n-1}} - \frac{(-1)^{n-1} \prod_{i=0}^{n-2} 2^{k_i}}{q_{n-1} q_{n-2}} \\ &= \frac{(-1)^{n-1} \prod_{i=0}^{n-2} 2^{k_i}}{q_n q_{n-2}} \left(\frac{-2^{k_{n-1}} q_{n-2}}{q_{n-1}} + \frac{q_n}{q_{n-1}} \right) \\ &= \frac{(-1)^{n-1} \prod_{i=0}^{n-2} 2^{k_i}}{q_n q_{n-2}} \cdot \frac{-2^{k_{n-1}} q_{n-2} + 2^{k_n} q_{n-1} + 2^{k_{n-1}} q_{n-2}}{q_{n-1}} \\ &= 2^{k_n} \frac{(-1)^{n-1} \prod_{i=0}^{n-2} 2^{k_i}}{q_n q_{n-2}}\end{aligned}$$

Since every p_n and q_n is a positive integer, we get from this result

$$\frac{p_{2n}}{q_{2n}} - \frac{p_{2n-2}}{q_{2n-2}} = 2^{k_{2n}} \frac{(-1)^{2n-1} \prod_{i=0}^{2n-2} 2^{k_i}}{q_{2n} q_{2n-2}} < 0$$

and

$$\frac{p_{2n+1}}{q_{2n+1}} - \frac{p_{2n-1}}{q_{2n-1}} = 2^{k_{2n+1}} \frac{(-1)^{2n} \prod_{i=0}^{2n-1} 2^{k_i}}{q_{2n+1} q_{2n-1}} > 0,$$

which proves monotonicity and orientation of the two sequences.

Furthermore, since $\frac{p_{2n+1}}{q_{2n+1}} - \frac{p_{2n}}{q_{2n}} = \frac{(-1)^{2n} \prod_{i=0}^{2n} 2^{k_i}}{q_{2n+1} q_{2n}} > 0$ for all n, the monotonicity of the two sequences ensures $\frac{p_{2m+1}}{q_{2m+1}} - \frac{p_{2n}}{q_{2n}} > 0$ for arbitrary m and n.

Finally, since $\lim_{n\to\infty} \frac{p_n}{q_n} = x$ then also the sequences of odd and even order convergents converge to x. Therefore $\frac{p_{2n}}{q_{2n}} \le x \le \frac{p_{2n+1}}{q_{2n+1}}$ holds for all n. □

Table 2.3: CCL convergents of $x = 100/2.54$ and their error from exact value.

i	k_i	p_i	q_i	$\frac{p_i}{q_i}$	$\lvert\frac{p_i}{q_i} - x\rvert$	$\lvert\frac{p_i}{q_i} - \frac{p_{i-1}}{q_{i-1}}\rvert$
-1	-	1	0	∞	∞	∞
0	5	32	1	32.00	7.4e-0	∞
1	2	40	1	40.00	6.2e-1	8.0e-0
2	3	352	9	39.11	2.6e-1	8.9e-1
3	1	512	13	39.38	1.5e-2	2.8e-1
4	3	4 448	113	39.36	7.2e-3	2.2e-2
5	0	8 544	217	39.37	3.2e-3	1.1e-2
6	0	6 496	165	39.37	3.9e-4	3.6e-3
7	2	60 512	1 537	39.37	1.2e-4	5.1e-4
8	1	86 496	2 197	39.37	2.9e-5	1.6e-4
9	1	233 504	5 931	39.37	1.1e-5	4.0e-5
10	1	5 000	127	39.37	0	1.1e-5

3 Continued Logarithm Arithmetic

The preceding section showed that continued logarithms have qualified for legal real-number representation; well, almost legal because of the limitation to $[1, \infty)$. This section will extend the previous results in direction of using continued logarithms for purposes of computer arithmetic.

Khinchin propagated continued fractions as a representation of great theoretical potential as *"they reproduce in a pure form the properties of the number that they represent"* [Khinchin, 1964]. At the same time, he conjectured that continued fractions are of limited practical application because of missing simple rules for arithmetic operations. However, Gosper showed in [Gosper, 1972, Gosper, 1978] that there is an algorithm for computing all basic operations in a uniform way. Moreover, such an algorithm is simple enough

Table 2.4: Convergents of regular cont. frac. of $x = 100/2.54$ and their error from exact value. For analytical formulas refer to [Khinchin, 1964].

i	a_i	p_i	q_i	$\frac{p_i}{q_i}$	$\lvert\frac{p_i}{q_i} - x\rvert$	$\lvert\frac{p_i}{q_i} - \frac{p_{i-1}}{q_{i-1}}\rvert$
-1	-	1	0	∞	∞	∞
0	39	39	1	39.00	3.7e-01	∞
1	2	79	2	39.50	1.3e-01	5.0e-01
2	1	118	3	39.33	3.7e-02	1.7e-01
3	2	315	8	39.38	4.9e-03	4.2e-02
4	2	748	19	39.37	1.7e-03	6.6e-03
5	1	1 063	27	39.37	2.9e-04	1.9e-03
6	4	5 000	127	39.37	0	2.9e-04

even for manual computation, thus legally useful for computational practice. Since then, those principles have been quickly adopted and generalized also for other representations [Potts, 1998, Edalat & Heckmann, 2002, Niqui, 2007, Kornerup & Matula, 1990].

3.1 Basic Principles

The fundamental idea published by Gosper in [Gosper, 1972] evolved from understanding of continued fractions as a LFT representation. Because of that, any rational transformation of such representation does not change its overall character. That is

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \prod \begin{pmatrix} a_i & b_i \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} a' & b' \\ c' & d' \end{pmatrix} = \prod \begin{pmatrix} a'_j & b'_j \\ 1 & 0 \end{pmatrix}.$$

On this basis, one can compute an arbitrary rational function of a single variable just by setting coefficients a, b, c, d appropriately.

The whole principle is basically very simple as it is nothing else than applying digit transformations $\left(\begin{smallmatrix} a_i & b_i \\ 1 & 0 \end{smallmatrix}\right)$ on the computed LFT one by one, in a left-to-right direction. Once we get a sufficiently "cumulated" LFT, we use inverse transformations $\left(\begin{smallmatrix} 0 & -b'_j \\ -1 & a'_j \end{smallmatrix}\right)$ to factorize it back into a continued fraction representation. Using a notation $\cdot^{(m;n)}$ to denote that $\cdot$ has been affected by n input digit transformations and m output inverse digit transformations, we may express the process as follows:

$$\begin{aligned} \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \prod_{i=0} \begin{pmatrix} a_i & b_i \\ 1 & 0 \end{pmatrix} &= \begin{pmatrix} a^{(0;n)} & b^{(0;n)} \\ c^{(0;n)} & d^{(0;n)} \end{pmatrix} \cdot \prod_{i=n} \begin{pmatrix} a_i & b_i \\ 1 & 0 \end{pmatrix} \\ &= \prod_{j=0}^{m-1} \begin{pmatrix} a'_j & b'_j \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} a^{(m;n)} & b^{(m;n)} \\ c^{(m;n)} & d^{(m;n)} \end{pmatrix} \cdot \prod_{i=n} \begin{pmatrix} a_i & b_i \\ 1 & 0 \end{pmatrix}. \end{aligned}$$

The only issue with this principle is to determine the elements of the factorization so that they respect the constraints of the particular continued fraction class.

Example 12. *Let us first demonstrate the previous principle in practice on CCL representation. We will compute first few elements of $\phi(x) = 2 + \frac{1}{x}$ for $x = \sqrt{10}$, expansion of which has been determined in Example 3. Thus $x = (1, 0, 0, 1, 1, 0, 1, 1, 0, 1, \ldots)_{CCL}$.*

The process of evaluating first few elements of the result is summarized in Tab. 3.1. The table is indexed by pairs of $(m; n)$, for which actual state of $\phi^{(m;n)}$ is provided. Starting from $\phi^{(0;0)} = \phi$, we change the actual state of $\phi^{(m;n)}$ between rows by taking "in" or "out" transformations, depending on whether an element of the argument x or result $\phi(x)$ is being processed. The two

transformations correspond to Eq. (3.1) and (3.2) respectively:

$$\phi^{(m;n)} \circ \kappa_{k_n} = \phi^{(m;n+1)}, \tag{3.1}$$

$$\kappa_{k'_m}{}^{-1} \circ \phi^{(m;n)} = \phi^{(m+1;n)}. \tag{3.2}$$

Columns for k_n and k'_m list elements that take part in the corresponding transformation. They refer to the elements of the argument and the result, respectively. Together with each $\phi^{(m;n)}$, we also provide its range over $\mathbb{I} = [1, \infty]$ to easily estimate whether an output of k'_m is possible. The choice of $\mathbb{I}$ comes from Section 2.4.

For illustration, we show samples of input and output transformations for values $(m; n)$ of $(0; 0)$ and $(0; 4)$, respectively. The input transformation is quite straightforward—we take the element k_0 and transform the $\phi^{(0;0)}$ accordingly. The output is straightforward as well, but we must first examine the range of $\phi^{(m;n)}$, whether its range satisfies condition $\phi^{(m;n)}(\mathbb{I}) \subseteq [2^{k'_m}, 2^{k'_m+1})$ for some k'_m. In our case, $\phi^{(0;4)}(\mathbb{I})$ fits within the interval $[2, 4)$, which corresponds to $k'_0 = 1$. The output transformation then uses a corresponding pseudo-inverse transformation.

For simplicity, LFTs listed in Tab. 3.1 have negative signs factored out.

$$k_0 = 1 \quad \rightarrow \quad \phi^{(0;1)} = \phi^{(0;0)} \circ \kappa_{k_0} \cong \left(\begin{smallmatrix} 2 & 1 \\ 1 & 0 \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} 2 & 2 \\ 1 & 0 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 5 & 4 \\ 2 & 2 \end{smallmatrix}\right)$$

$$\phi^{(0;4)}(\mathbb{I}) \subseteq [2.31, 2.33] \quad \rightarrow \quad k'_0 = 1 \quad \rightarrow \quad \phi^{(1;4)} = \kappa_{k'_0}{}^{-1} \circ \phi^{(0;4)} \cong \left(\begin{smallmatrix} 0 & -2 \\ -1 & 2 \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} 37 & 28 \\ 16 & 12 \end{smallmatrix}\right) = \left(\begin{smallmatrix} -32 & -24 \\ -5 & -4 \end{smallmatrix}\right)$$

Table 3.1: Summary of a process computing $\phi(x) = 2 + \frac{1}{x}$ for $x = \sqrt{10}$.

m	n	$\phi^{(m;n)}$	$\phi^{(m;n)}(\mathbb{I})$	k_n	k'_m	Transf.
0	0	$\left(\begin{smallmatrix} 2 & 1 \\ 1 & 0 \end{smallmatrix}\right)$	$\left[\frac{2}{1}, \frac{3}{1}\right] \subseteq [2.0, 3.0]$	1		in
0	1	$\left(\begin{smallmatrix} 5 & 4 \\ 2 & 2 \end{smallmatrix}\right)$	$\left[\frac{9}{4}, \frac{5}{2}\right] \subseteq [2.2, 2.5]$	0		in
0	2	$\left(\begin{smallmatrix} 9 & 5 \\ 4 & 2 \end{smallmatrix}\right)$	$\left[\frac{9}{4}, \frac{14}{6}\right] \subseteq [2.2, 2.4]$	0		in
0	3	$\left(\begin{smallmatrix} 14 & 9 \\ 6 & 4 \end{smallmatrix}\right)$	$\left[\frac{23}{10}, \frac{14}{6}\right] \subseteq [2.3, 2.4]$	1		in
0	4	$\left(\begin{smallmatrix} 37 & 28 \\ 16 & 12 \end{smallmatrix}\right)$	$\left[\frac{37}{16}, \frac{65}{28}\right] \subseteq [2.3, 2.4]$		1	out
1	4	$\left(\begin{smallmatrix} 32 & 24 \\ 5 & 4 \end{smallmatrix}\right)$	$\left[\frac{56}{9}, \frac{32}{5}\right] \subseteq [6.2, 6.4]$		2	out
2	4	$\left(\begin{smallmatrix} 5 & 4 \\ 3 & 2 \end{smallmatrix}\right)$	$\left[\frac{5}{3}, \frac{9}{5}\right] \subseteq [1.6, 1.8]$		0	out
3	4	$\left(\begin{smallmatrix} 3 & 2 \\ 2 & 2 \end{smallmatrix}\right)$	$\left[\frac{5}{4}, \frac{3}{2}\right] \subseteq [1.2, 1.5]$		0	out
4	4	$\left(\begin{smallmatrix} 2 & 2 \\ 1 & 0 \end{smallmatrix}\right)$	$\left[\frac{2}{2}, \frac{4}{2}\right] \subseteq [2.0, 4.0]$	1		in
4	5	$\left(\begin{smallmatrix} 6 & 4 \\ 2 & 2 \end{smallmatrix}\right)$	$\left[\frac{10}{4}, \frac{6}{2}\right] \subseteq [2.5, 3.0]$			...
...	...	...	...	...	...	...

The computed result is $\phi(x) = (1, 2, 0, 0, \ldots)_{CCL}$. Let us verify, if it is correct. For this purpose, we evaluate a range $\mathbb{D}$ of numbers that have $(1, 2, 0, 0)_{CCL}$ as their prefix. That is:

$$\begin{aligned} \mathbb{D} &= \kappa_{k_0} \circ \kappa_{k_1} \circ \kappa_{k_2} \circ \kappa_{k_3}(\mathbb{I}) = \kappa_1 \circ \kappa_2 \circ \kappa_0 \circ \kappa_0(\mathbb{I}) \\ &\cong K_1 \cdot K_2 \cdot (K_0)^2 \cdot \begin{pmatrix} \mathbb{I} \\ 1 \end{pmatrix} = \begin{pmatrix} 28 & 18 \\ 12 & 8 \end{pmatrix} \cdot \begin{pmatrix} \mathbb{I} \\ 1 \end{pmatrix} \\ &\subseteq \left[\frac{23}{10}, \frac{7}{3}\right] \subseteq [2.30, 2.34). \end{aligned}$$

Since $2 + \frac{1}{\sqrt{10}} = 2.316\ldots$, we see that the computed CCL prefix represents quite well the actual value of $\phi(\sqrt{10})$. □

On the basis of the example, we can identify three core steps in computing a LFT:

Algorithm 1: Process of computing k output elements of a LFT $\phi(x)$.

```
Input: φ – a LFT given as a four-tuple of its coefficients.
Input: x – an argument in a LFT representation.
Input: k – required length of φ(x)'s expansion.
Output: Expansion of φ(x).
m = n = 0;
𝔻_m = range(x,n);
e = ∅;
while (φ(𝔻_m) ⊈ {∞} ∧ m < k) do
    e_m = decide(φ, 𝔻_m);
    if (e_m ≠ ∅) then
        φ = emit(φ,e_m);
        e = e ∘ e_m;
        m = m + 1;
        continue;
    end
    φ = absorb(φ,x,n);
    n = n + 1;
    𝔻_m = range(x,n);
end
return e;
```

- *Absorption* (or *consumption*): An input phase, wherein we take successive elements of the argument and transform the computed LFT accordingly. Be $\phi^{(m;n)}$ the actual state of the computed LFT, absorption of $(n+1)$-th element changes this state to $\phi^{(m;n+1)}$.
- *Estimation* and *decision*: An intermediate phase, wherein we determine range $\phi^{(m;n)}(\mathbb{I})$ of an actual state of the computed LFT over a base domain $\mathbb{I}$ so as to see whether an output element could be determined. The base domain is specific for each continued fraction class, e.g. $[1, \infty]$ for continued logarithms.
- *Emission*: An output phase, wherein an element associated with the range of the computed LFT is emitted, transforming its actual state $\phi^{(m;n)}$ accordingly into $\phi^{(m+1;n)}$. The performed transformation is inverse to the one associated with the output element.

These steps may be arbitrarily interleaved, except for emission that may occur only if decision finds an element to output. Commonly, however, the steps are ordered in a *lazy evaluation scheme* known from functional languages. This is a data-driven evaluation, in which elements of the result are output until the computed result satisfies a user-defined constraint (e.g. required precision). If emission is not possible, the absorption occurs as long as decision does not signal a positive output. This process is shown in Alg. 1.

The algorithm is rather general and its core functions `range`, `decide`, `absorb` and `emit` need to be specified for each particular representation. The `absorb` and `emit` functions perform the input and output transformations—they take the four integer variables a, b, c, d representing the actual state $\phi^{(m;n)}$ of the LFT and adapt them appropriately to the given element. The `range` function returns the interval associated with the yet unprocessed continuation of the input argument. Finally, the `decide` function performs the range estimation of $\phi^{(m;n)}(\mathbb{D}_m)$ and decides whether an output element could be emitted.

Example 12 provided clues on how to construct the individual functions for CCL. Therefore in the following, we will focus on BCL representation. Absorption and emission transformations are obvious, but for completeness their overview is given in Tab. 3.2. The `range` of input argument is also easy; indeed equal to that of CCL—it returns the base domain $\mathbb{I} = [1, \infty]$, or $\{\infty\}$ if the input argument has terminated.

Table 3.2: Input and output LFT transformations for BCL.

Step	Formal expression	LFT transformation	
Absorption	$\phi^{(m;n)} \circ \beta_{b_n}$	$b_n = 1$	$\left(\begin{smallmatrix} 2a & b \\ 2c & d \end{smallmatrix}\right)$
		$b_n = 0$	$\left(\begin{smallmatrix} a+b & a \\ c+d & c \end{smallmatrix}\right)$
Emission	$\beta^{-1}_{b'_m} \circ \phi^{(m;n)}$	$b'_m = 1$	$\left(\begin{smallmatrix} a & b \\ 2c & 2d \end{smallmatrix}\right)$
		$b'_m = 0$	$\left(\begin{smallmatrix} c & d \\ a-c & b-d \end{smallmatrix}\right)$

The `decide` function is likely the most complex one. We already know it performs two tasks: range estimation of $\phi^{(m;n)}$ and evaluation of output conditions. Since $\phi^{(m;n)}$ is a LFT, range estimation is achieved by inspecting its values at the borders of the assumed domain $\mathbb{D}_m$, but $\phi^{(m;n)}$ must be *well-defined* on it (i.e. continuous and monotone). By its nature, well-definedness depends on sign change of the LFT's denominator over the assumed domain. Thus, if $\phi^{(m;n)} \cong \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$, Eq. (3.3) is a sufficient condition of well-definedness[7]. Notice that this discussion makes sense only if $\mathbb{D}_m = \mathbb{I}$; otherwise, if $\mathbb{D}_m = \{\infty\}$, $\phi^{(m;n)}$ collapses to a rational number $\frac{a}{c}$.

$$0 \neq \operatorname{sgn}(c) = \operatorname{sgn}(c+d), \tag{3.3}$$

where the signum operator $\operatorname{sgn}(x)$ returns $-1, 0, +1$ as usual. For a well-defined LFT, inspection of output conditions follows Def. 4:

$$\begin{aligned} \tfrac{a}{c} \in [2,\infty] \ \wedge \ \tfrac{a+b}{c+d} \in [2,\infty] \quad &\Rightarrow \quad b'_m = 1, \\ \tfrac{a}{c} \in [1,2) \ \wedge \ \tfrac{a+b}{c+d} \in [1,2) \quad &\Rightarrow \quad b'_m = 0. \end{aligned}$$

Example 13. *Using Alg. 1, let us compute first few elements of $\phi(x) = 2 + \frac{1}{x}$ for $x = \sqrt{10}$ in a BCL representation. Thanks to the equivalence of BCL and CCL, we can directly rewrite the result of Example 3: $x = (1000101001010010\ldots)_{BCL}$.*

Progress of Alg. 1 for this argument and $\phi \cong \left(\begin{smallmatrix} 2 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ is provided in Tab. 3.3. The table is indexed by the couple $(m;n)$ and lists figures of $\phi^{(m;n)}$ and e_m taken at line 5 of the algorithm; consider that e_m stands for a bit b'_m of the result. Since x is irrational, $\mathbb{D}_m = \mathbb{I}$ all the time and is thus omitted from the figures. On the other hand, the table provides actual values of $\phi^{(m;n)}(1) = \frac{a+b}{c+d}$ and $\phi^{(m;n)}(\infty) = \frac{a}{c}$ for easy decision on output symbol.

The computed result is $(10110001\ldots)_{BCL}$ and, as expected, it corresponds to the result of Example 12.

We should also remark on one practical fact at this point. Notice that $\phi^{(3;0)} \cong \left(\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right)$ is indeed the identity. It means that we could spare any further effort on computing and could automatically forward input bits to the output. □

3.2 Binary Operations

Following the principles above, we employ BLFTs to compute binary operations. Again, the BLFT is general enough to compute any rational transformation of two variables, including addition, subtraction, multiplication and division, or their combination.

Example 14. *Let us compute $\phi(x,y) = \frac{2xy+x}{xy}$ for CCL representation and $x = y = \sqrt{10} = (1,0,0,1,1,0,1,1,0,1,\ldots)_{CCL}$. The result is expected to equal that of Example 12.*

First of all, we need to derive how absorption and emission affects the BLFT $\phi^{(m;n,o)} \cong \left(\begin{smallmatrix} a & b & c & d \\ e & f & g & h \end{smallmatrix}\right)$ that represents the actual state of computation. Again, m counts the number of

[7]Notice that with current representation, we can compute only LFTs mapping $\mathbb{I}$ to itself or its subset. This automatically ensures well-definedness, but we will need Eq. (3.3) later for extended representations in Section 4 and Section 5.

Table 3.3: A snapshot of variables and parameters taken at line 5 of Alg. 1 computed for BCL.

m	n	$\phi^{(m;n)}$	e_m	$\frac{a}{c}$	$\frac{a+b}{c+d}$
0	0	$\left(\begin{smallmatrix}2&1\\1&0\end{smallmatrix}\right)$	1	2	3
1	0	$\left(\begin{smallmatrix}2&1\\2&0\end{smallmatrix}\right)$	0	1	$\frac{3}{2}$
2	0	$\left(\begin{smallmatrix}2&0\\0&1\end{smallmatrix}\right)$	1	∞	2
3	0	$\left(\begin{smallmatrix}2&0\\0&2\end{smallmatrix}\right)$	$\emptyset$	∞	1
3	1	$\left(\begin{smallmatrix}4&0\\0&2\end{smallmatrix}\right)$	1	∞	2
4	1	$\left(\begin{smallmatrix}4&0\\0&4\end{smallmatrix}\right)$	$\emptyset$	∞	1
4	2	$\left(\begin{smallmatrix}4&4\\4&0\end{smallmatrix}\right)$	$\emptyset$	1	2
4	3	$\left(\begin{smallmatrix}8&4\\4&4\end{smallmatrix}\right)$	$\emptyset$	2	$\frac{3}{2}$
4	4	$\left(\begin{smallmatrix}12&8\\8&4\end{smallmatrix}\right)$	0	$\frac{3}{2}$	$\frac{5}{3}$
5	4	$\left(\begin{smallmatrix}8&4\\4&4\end{smallmatrix}\right)$	$\emptyset$	2	$\frac{3}{2}$
6	5	$\left(\begin{smallmatrix}16&4\\8&4\end{smallmatrix}\right)$	$\emptyset$	2	$\frac{5}{3}$
6	6	$\left(\begin{smallmatrix}20&16\\12&8\end{smallmatrix}\right)$	0	$\frac{5}{3}$	$\frac{9}{5}$
7	6	$\left(\begin{smallmatrix}12&8\\8&8\end{smallmatrix}\right)$	0	$\frac{4}{3}$	$\frac{5}{4}$
8	6	$\left(\begin{smallmatrix}8&8\\4&0\end{smallmatrix}\right)$	1	2	4
...	...	...	...	...	...

emissions, while n and o respectively count the number of absorptions from x and y. Emission/absorption transformations follow the pattern of LFT composition. Emission then goes as follows:

$$\phi^{(m+1;n,o)} = \kappa_{k'_m}^{-1} \circ \phi^{(m;n,o)} \cong \begin{pmatrix} e2^{k'_m} & f2^{k'_m} & g2^{k'_m} & h2^{k'_m} \\ a - e2^{k'_m} & b - f2^{k'_m} & c - g2^{k'_m} & d - h2^{k'_m} \end{pmatrix} .$$

Absorption can take input either from x, or y, transforming the BLFT as below. We use k_n and k_o to denote n-th and o-th input element of x and y respectively.

$$\phi^{(m;n+1,o)}(x_{n+1}, y_o) = \phi^{(m;n,o)}(\kappa_{k_n}(x_{n+1}), y_o) \cong \begin{pmatrix} a2^{k_n} + b & a2^{k_n} & c2^{k_n} + d & c2^{k_n} \\ e2^{k_n} + f & e2^{k_n} & g2^{k_n} + h & e2^{k_n} \end{pmatrix} \cdot \begin{bmatrix} x_{n+1} \\ y_o \end{bmatrix} ,$$

$$\phi^{(m;n,o+1)}(x_n, y_{o+1}) = \phi^{(m;n,o)}(x_n, \kappa_{k_o}(y_{o+1})) \cong \begin{pmatrix} a2^{k_o} + c & b2^{k_o} + d & a2^{k_o} & b2^{k_o} \\ e2^{k_o} + g & f2^{k_o} + h & e2^{k_o} & f2^{k_o} \end{pmatrix} \cdot \begin{bmatrix} x_n \\ y_{o+1} \end{bmatrix} ,$$

where $\left[\begin{smallmatrix}x\\y\end{smallmatrix}\right]$ stands for a transposed vector $\left(\begin{smallmatrix}xy & y & x & 1\end{smallmatrix}\right)$.

Knowing the transformations, we can approach to computation itself. We will first absorb few elements of both arguments and then examine the possibility of output. The first two absorptions are shown here:

$$\phi^{(0;0,0)} \cong \left(\begin{smallmatrix}2&1&0&0\\1&0&0&0\end{smallmatrix}\right) \quad \rightarrow \quad k_{x_0} = 1 \quad \rightarrow \quad \phi^{(0;1,0)} \cong \left(\begin{smallmatrix}5&4&0&0\\2&2&0&0\end{smallmatrix}\right)$$
$$\phi^{(0;1,0)} \cong \left(\begin{smallmatrix}5&4&0&0\\2&2&0&0\end{smallmatrix}\right) \quad \rightarrow \quad k_{y_0} = 1 \quad \rightarrow \quad \phi^{(0;1,1)} \cong \left(\begin{smallmatrix}10&8&10&8\\4&4&4&4\end{smallmatrix}\right)$$

Going on like this, we would get $\phi^{(0;4,4)} \cong \left(\begin{smallmatrix}148&112&111&84\\64&48&48&36\end{smallmatrix}\right)$ after canceling all common factors. This result is irrespective of the order of interleaving absorptions from x and y.

Estimating range of a BLFT is similar to that of LFTs, except that the definition domain is now two dimensional. As both arguments are within $\mathbb{I}$, the domain of our interest is $\mathbb{I} \times \mathbb{I}$. Since our BLFT is well-defined on that domain, it is enough to examine BLFT's value at the four corners $(x, y) \in \{(1, 1), (1\infty), (\infty, 1), (\infty, \infty)\}$. Thus $\phi^{(0;4,4)}(\mathbb{I}, \mathbb{I}) \subseteq [2.31, 2.33]$, indicating that we can emit $k'_0 = 1$. Further emission is possible as shown on the sample emission below. More complete results are provided in Tab. 3.4.

$$\phi^{(0;4,4)}(\mathbb{I}, \mathbb{I}) \subseteq [2.31, 2.33] \quad \rightarrow \quad k'_0 = 1 \quad \rightarrow \quad \phi^{(1;4,4)} \cong \left(\begin{smallmatrix}128&96&96&72\\20&16&15&12\end{smallmatrix}\right) ,$$
$$\phi^{(1;4,4)}(\mathbb{I}, \mathbb{I}) \subseteq [6.22, 6.40] \quad \rightarrow \quad k'_1 = 2 \quad \rightarrow \quad \phi^{(2;4,4)} \cong \left(\begin{smallmatrix}20&16&15&12\\12&8&9&6\end{smallmatrix}\right) .$$

Table 3.4: Results of computing $\phi(x,y) = \frac{2xy+x}{xy}$ for CCL and $x = y = \sqrt{10}$.

m	n	o	$\phi^{(m;n,o)}$	k'_m	k_n	k_o	$\phi^{(m;n,o)}(\mathbb{I} \times \mathbb{I})$
0	0	0	$\left(\begin{smallmatrix} 2 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{smallmatrix}\right)$		1		$[2.00, 3.00]$
0	1	0	$\left(\begin{smallmatrix} 5 & 4 & 0 & 0 \\ 2 & 2 & 0 & 0 \end{smallmatrix}\right)$			1	$[2.25, 2.50]$
0	1	1	$\left(\begin{smallmatrix} 5 & 4 & 5 & 4 \\ 2 & 2 & 2 & 2 \end{smallmatrix}\right)$		0		$[2.25, 2.50]$
0	2	1	$\left(\begin{smallmatrix} 9 & 5 & 9 & 5 \\ 4 & 2 & 4 & 2 \end{smallmatrix}\right)$			0	$[2.25, 2.33]$
0	2	2	$\left(\begin{smallmatrix} 18 & 10 & 9 & 5 \\ 8 & 4 & 4 & 2 \end{smallmatrix}\right)$		0		$[2.25, 2.33]$
0	3	2	$\left(\begin{smallmatrix} 28 & 18 & 14 & 9 \\ 12 & 8 & 6 & 4 \end{smallmatrix}\right)$			0	$[2.30, 2.33]$
0	3	3	$\left(\begin{smallmatrix} 42 & 27 & 28 & 18 \\ 18 & 12 & 12 & 8 \end{smallmatrix}\right)$		1		$[2.30, 2.33]$
0	4	3	$\left(\begin{smallmatrix} 111 & 84 & 74 & 56 \\ 48 & 36 & 32 & 24 \end{smallmatrix}\right)$			1	$[2.32, 2.33]$
0	4	4	$\left(\begin{smallmatrix} 148 & 112 & 111 & 84 \\ 64 & 48 & 48 & 36 \end{smallmatrix}\right)$	1			$[2.32, 2.33]$
1	4	4	$\left(\begin{smallmatrix} 128 & 96 & 96 & 72 \\ 20 & 16 & 15 & 12 \end{smallmatrix}\right)$	2			$[6.22, 6.40]$
2	4	4	$\left(\begin{smallmatrix} 20 & 16 & 15 & 12 \\ 12 & 8 & 9 & 6 \end{smallmatrix}\right)$	0			$[1.66, 1.80]$
3	4	4	$\left(\begin{smallmatrix} 12 & 8 & 9 & 6 \\ 8 & 8 & 6 & 6 \end{smallmatrix}\right)$	0			$[1.25, 1.50]$
4	4	4	$\left(\begin{smallmatrix} 8 & 8 & 6 & 6 \\ 4 & 0 & 3 & 0 \end{smallmatrix}\right)$				$[2.00, 4.00]$
...	...	...	...	...	...	...	...

□

The use of BLFTs copies that of LFTs, but it takes on some complexity to handle twice more the number of input variables. This is especially the case of decision process (discussed below), but also affects e.g. absorption since we have to choose which argument to input (see Section 3.3).

Some complexity is also added by the fact that we have to control eight integer variables, arranged in a two-by-four matrix, for which we can hardly apply simple matrix transformations. There is though a chance to use matrix operations, but in less direct form than for LFTs. If $A = \left(\begin{smallmatrix} a & b & c & d \\ e & f & g & h \end{smallmatrix}\right)$ is a BLFT and $D = \left(\begin{smallmatrix} p & q \\ r & s \end{smallmatrix}\right)$ is a digit matrix to be absorbed in or emitted out of that BLFT, then the transformations are obtained as follows:

$$\begin{array}{ll} \text{Emission:} & D^* \cdot A, \\ x\text{-absorption:} & A \cdot X, \\ y\text{-absorption:} & A \cdot Y, \end{array}$$

where X and Y are derived from D as

$$X = \begin{pmatrix} p & q & 0 & 0 \\ r & s & 0 & 0 \\ 0 & 0 & p & q \\ 0 & 0 & r & s \end{pmatrix} \quad \text{and} \quad Y = \begin{pmatrix} p & 0 & q & 0 \\ 0 & p & 0 & q \\ r & 0 & s & 0 \\ 0 & r & 0 & s \end{pmatrix}.$$

However, precomputing the whole set of transformations (as in Example 14) and using a look-up table approach seems a better way to automate the process. Such a look-up table for BCL is in Tab. 3.5.

Table 3.5: Transformations of fractional coefficients of a BLFT with respect to an absorbed/emitted BCL bit.

Bit	x-absorption	y-absorption	Emission
0	$\left(\begin{smallmatrix} a+b & a & c+d & c \\ e+f & e & g+h & g \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} a+c & b+d & a & b \\ e+g & f+h & e & f \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} e & f & g & h \\ a-e & b-f & c-g & d-h \end{smallmatrix}\right)$
1	$\left(\begin{smallmatrix} 2a & b & 2c & d \\ 2e & f & 2g & h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2a & 2b & c & d \\ 2e & 2f & g & h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} a & b & c & d \\ 2e & 2f & 2g & 2h \end{smallmatrix}\right)$

As indicated, the extension to two dimensions affected especially the decision process. The range evaluation now happens over a cartesian product of a domain assumed for input

Algorithm 2: Process of computing k output elements of a BLFT $\phi(x, y)$.

```
Input: φ – a BLFT given as an eight-tuple of its coefficients.
Input: x, y – arguments in cont. log. representation.
Input: k – required length of φ(x,y)'s expansion.
Output: Cont. log. expansion of φ(x,y).
m = n = o = 0;
𝔻_m = range(x,n)×range(y,o);
e = ∅;
while (φ(𝔻_m) ⊈ {∞} ∧ m < k) do
    e_m = decide(φ, 𝔻_m);
    if (e_m ≠ ∅) then
        φ = emit(φ,e_m);
        e = e ∘ e_m;
        m = m + 1;
        continue;
    end
    𝕊 = select(x, n, y, o);
    if x ∈ 𝕊 then
        φ = absorb(φ,x,n);
        n = n + 1;
    end
    if y ∈ 𝕊 then
        φ = absorb(φ,y,o);
        o = o + 1;
    end
    𝔻_m = range(x,n)×range(y,o);
end
return e;
```

arguments—i.e. $\mathbb{D} = \mathbb{I} \times \mathbb{I}$ for continued logarithms. For a well-defined mapping $\mathbb{D} \to \mathbb{R}$, one should thus examine the mapping's values at the borders of $\mathbb{D}$. Fortunately, thanks to the linear character of BLFTs, this examination further restricts to only the four corners of $\mathbb{D}$. Let fractions $\frac{n_i}{d_i}$ represent the values of a BLFT in those corners; if $A = \left(\begin{smallmatrix} a & b & c & d \\ e & f & g & h \end{smallmatrix}\right)$ is a BLFT, then:

$$\begin{aligned} n_0 &= a, & n_1 &= a + b, & n_2 &= a + c, & n_3 &= a + b + c + d, \\ d_0 &= e, & d_1 &= e + f, & d_2 &= e + g, & d_3 &= e + f + g + h. \end{aligned} \tag{3.4}$$

A sufficient condition of BLFTs' well-definedness is then

$$\operatorname{sgn}(d_0) = \operatorname{sgn}(d_1) = \operatorname{sgn}(d_2) = \operatorname{sgn}(d_3) \neq 0, \tag{3.5}$$

meaning that the denominator of the BLFT does not change its sign over $\mathbb{I} \times \mathbb{I}$, nor does it equal zero.

Domain $\mathbb{D}$ changes with terminating one or both input arguments. E.g. consider that y has terminated; assuming $y = \infty$, the domain shrinks to $\mathbb{D} = \mathbb{I} \times \{\infty\}$ and BLFT $\left(\begin{smallmatrix} a & b & c & d \\ e & f & g & h \end{smallmatrix}\right)$ restricts to a simple LFT in the variable x, i.e. $\left(\begin{smallmatrix} a & b \\ e & f \end{smallmatrix}\right)$. The well-definedness condition changes appropriately, in this case ignoring d_2 and d_3—we could, however, let $c = d = g = h = 0$ obtaining $d_0 = d_2$ and $d_1 = d_3$, which would have the same effect.

Considering all that has been said above, we could adapt Alg. 1 to Alg. 2. Likely the only change requiring closer explanation is the function `select`. It returns a set of arguments to absorb from, giving it chance to realize absorption from either one or both arguments at a time.

Example 15. *Let us compute first few elements of $\phi(x, y) = xy$ for $x = y = \sqrt{10}$ in a BCL representation. Recall that $\sqrt{10} = (1000101001010010\ldots)_{BCL}$. Similarly to Example 14,*

Table 3.6: A snapshot of variables and parameters taken at line 5 of Alg. 2 computed for BCL and $\phi = xy$ with $x = y = \sqrt{10}$.

m	n	o	$\phi^{(m;n,o)}$	e_m	$\frac{n_0}{d_0}$	$\frac{n_1}{d_1}$	$\frac{n_2}{d_2}$	$\frac{n_3}{d_3}$
0	0	0	$\left(\begin{smallmatrix}1&0&0&0\\0&0&0&1\end{smallmatrix}\right)$	$\emptyset$	∞	∞	∞	1
0	1	0	$\left(\begin{smallmatrix}2&0&0&0\\0&0&0&1\end{smallmatrix}\right)$	$\emptyset$	∞	∞	∞	2
0	1	1	$\left(\begin{smallmatrix}4&0&0&0\\0&0&0&1\end{smallmatrix}\right)$	$\emptyset$	∞	∞	∞	4
0	2	1	$\left(\begin{smallmatrix}4&4&0&0\\0&0&1&0\end{smallmatrix}\right)$	$\emptyset$	∞	∞	4	8
0	2	2	$\left(\begin{smallmatrix}4&4&4&4\\1&0&0&0\end{smallmatrix}\right)$	1	4	8	8	16
1	2	2	$\left(\begin{smallmatrix}2&2&2&2\\1&0&0&0\end{smallmatrix}\right)$	1	2	4	4	8
2	2	2	$\left(\begin{smallmatrix}1&1&1&1\\1&0&0&0\end{smallmatrix}\right)$	$\emptyset$	1	2	2	4
2	3	2	$\left(\begin{smallmatrix}2&1&2&1\\1&1&0&0\end{smallmatrix}\right)$	$\emptyset$	2	$\frac{3}{2}$	4	3
2	3	3	$\left(\begin{smallmatrix}4&2&2&1\\1&1&1&1\end{smallmatrix}\right)$	1	4	3	3	$\frac{9}{4}$
3	3	3	$\left(\begin{smallmatrix}4&2&2&1\\2&2&2&2\end{smallmatrix}\right)$	$\emptyset$	2	$\frac{3}{2}$	$\frac{3}{2}$	$\frac{9}{8}$
3	4	3	$\left(\begin{smallmatrix}6&4&3&2\\4&2&4&2\end{smallmatrix}\right)$	0	$\frac{3}{2}$	$\frac{5}{3}$	$\frac{9}{8}$	$\frac{5}{4}$
4	4	3	$\left(\begin{smallmatrix}4&2&4&2\\2&2&-1&0\end{smallmatrix}\right)$	$\emptyset$	2	$\frac{3}{2}$	8	4
4	4	4	$\left(\begin{smallmatrix}8&4&4&2\\1&2&2&0\end{smallmatrix}\right)$	1	8	4	4	$\frac{18}{5}$
5	4	4	$\left(\begin{smallmatrix}4&2&2&1\\1&2&2&0\end{smallmatrix}\right)$	$\emptyset$	4	2	2	$\frac{9}{5}$
...	...	...	...	...	...	...	...	...

progress of Alg. 2 for assumed arguments and $\phi \cong \left(\begin{smallmatrix}1&0&0&0\\0&0&0&1\end{smallmatrix}\right)$ *is shown in Tab. 3.6. The table is indexed by the triple* $(m; n, o)$ *and list figures of* $\phi^{(m;n,o)}$ *and* e_m *taken at line 5 of the algorithm;* e_m *again stands for a bit* b'_m *of the result. In addition, we provide values of fractions* $\frac{n_i}{d_i}$ *as defined above. Domain* $\mathbb{D}_m = \mathbb{I} \times \mathbb{I}$ *all the time as a result of arguments' irrationality. The absorption scheme used for this example is the strict alternation, i.e. absorbing first from one argument and then from the other one, and repeating this alternation until some of the arguments terminates.*

The computed result is $(11101\ldots)_{BCL}$ *with an associated interval* $[8, 12]$*, which contains the expected value of 10.* □

3.3 Further Details

Purpose of this section is to discuss topics that closely relate to the principles of arithmetic introduced in this chapter. In particular, we will briefly discuss:

- Extension of LFT arithmetic to multiple variables,
- conversion between different LFT representations, and
- absorption schemes for binary (and multi-variable) arithmetic.

Multi-variable Operations

Following the shift from unary to binary LFTs, one could easily extend the LFT arithmetic principles to multiple (> 2) variables. Since these principles are completely regular and uniform in every variable, one could devise a universal algorithm that could compute an arbitrary multi-linear fractional transformation (n-LFT). This was indeed studied in [Nielsen, 1993].

The advantage of n-LFT arithmetic is introducing a general multi-variable operator that could handle all rational transformations up to n variables at once. The problem is, however, an exponential increase of complexity, which would in practice likely limit the n-LFT arithmetic to some small n. Whether or not this complexity is worth of such a computational "power" is yet to be investigated.

Conversion to/from Other LFT Representation

If we consider the general idea of Eq. (3.1), we can realize that it does not depend on the choice of input or output representation. That is, the input and output may not follow the same representation rules as we assumed so far.

This gives a chance to easily convert between any LFT-based representations. All it takes is to use absorption transformations relating to the input representation, and decision procedure and emission transformations relating to the output.

In context of continued logarithms, this result could be utilized in several ways. First of all, the conventional decimal representation has a LFT character. A number $x \in [0, 1]$, written as $x = 0.d_0 d_1 d_2 \ldots$ with $d_i \in \{0, 1, \ldots, 9\}$, could be expressed as $\prod \left(\begin{smallmatrix} 1 & d_i \\ 0 & 10 \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} 0 \\ 1 \end{smallmatrix}\right)$; with proper extension, one could cover entire real line. This result is indeed irrespective of the chosen radix. We can thus easily convert between continued logarithm and any conventional positional radix representation.

Secondly, a number of irrational numbers such as π, e and etc. have known expansions of simple continued fractions. By the general conversion mechanism, we may get their corresponding representation in a continued logarithm form.

Absorption Schemes

Absorption scheme refers to the `select` procedure of Alg. 2 choosing one or the other argument for absorption, or potentially both of them. Theoretically, the computed result is irrespective of the particular ordering of absorptions and emissions, but in practice the absorption scheme matters. It actually affects affairs such as the computational effort of evaluating input arguments, size of integer variables representing the computed (B)LFT, or frequency of emission. This problem was partly studied in [Potts, 1998], whereto one can refer for more details.

In practice, the most common absorption scheme is the *strict alternation*. In this scheme, with every absorption we change the role of the absorption source between the two arguments. Implementation of this scheme is fairly simple as it needs a single flag indicating which of the arguments is in turn to absorb from.

Another simple strategy, called *simultaneous*, is to absorb from both arguments at the same time. Its results would be similar to those of strict alternation, but it may require a bit more computational effort. Consider a situation, where we are about to emit the last element of the result, but need a further information from any of the input arguments. With strict alternation, we would absorb from the selected source, emit and finish. With simultaneous absorption, we would absorb from both arguments (i.e. two absorptions), emit and finish. Absorbing an input element may require evaluation of the corresponding expression subtree, effort of which would depend on the actual state of computation of that subtree—but such effort may be quite considerable. Therefore, simultaneous absorption seems less profitable than strict alternation. However, this theoretical conclusion would deserve an experimental proof.

None of the two preceding schemes considered the "information requirements" of the fractional transformation being computed. It may happen, that the actual state of the computation would benefit more from absorbing from one argument instead of the other one. Gosper identified such approach already in [Gosper, 1972]; Potts studied the principle called *information overlap strategy* [Potts, 1998]. While such an *information driven* scheme would yield the most

optimal results [Potts, 1998], its implementation does not seem simple enough to pay off.

4 Rational Extension

As we already know, continued logarithms can represent only numbers not lower than unity. To be capable of at least rational arithmetic, it is necessary to introduce some extension beyond the original definition. Inspired by the nature of the representation, it is clear that the extension will have a form of additional transformation(s) that shift the original value (negative or fractional between zero and unity) into the base domain $\mathbb{I} = [1, \infty)$.

In the following, we cover extension of the BCL representation first and then apply the derived results to CCL.

4.1 Binary Extension

The choice of transformation(s) should respect the original character of the BCL representation. That is:

- *Successive refinement*, which dictates the extension to have a form of prefix transformations. Composition of such transformations projects the original value into the base interval, where it is possible to use standard continued logarithm algorithms. Since the continued logarithm transformations are closed on the base interval, the prefix character of new transformations is natural. Using this approach is preferable because it does not need to adapt already defined continued logarithm algorithms.
- *Strict monotonicity* of every new transformation, which follows the uniqueness of the original representation and simplicity of relational functions.
- *Dense information entropy*, which is equivalent to requiring the volume of introduced additional transformations to be minimal. The original continued logarithm associated each digit with a certain transformation. In this sense, introducing new transformation extends the digit set, resulting in a need of representing each digit by multiple bits. The ideal number of new transformations is thus one or two.
- *Simplicity.* In order to follow the nature of original continued logarithm algorithms, every new transformation should be realized by shift-add/sub transformations. Ideally, the transformations should utilize as much as possible the same operations on the same operands, which are already in use in original algorithms (refer to Tab. 3.2 and 3.5). Meeting this requirement is especially important for hardware design, since it keeps low count of different data paths.

There are several ways to extend the representation and conform to the previous requirements. We will discuss three possible extensions, which we will call *natural*, *intuitive* and *symmetrical*. They are very similar and could be altogether formalized by the following definition. Fig. 4.1 visualizes the recursive relation of the individual extensions.

Definition 16. Extended binary continued logarithm *representation (EBCL) of a real number* x *is a sequence of* digits $\{b_i\}$, $b_i \in \{-, /, 0, 1\}$, $i = 0, 1, \ldots$, *such that there is a sequence* $\{x_i\}$, $x_{i>0} \in \mathbb{R}^* \setminus \{0\}$ *and* $x_0 = x$, *satisfying*

$$x_{i+1} = \beta_{b_i}(x_i) \qquad \textit{and} \qquad b_i = c$$

for a $c \in \{-, /, 0, 1\}$ *such that* $x_i \in \mathbb{B}_c$, *where*

$$\mathbb{B}_- = (\infty, 0), \qquad \mathbb{B}_/ = [0, 1), \qquad \mathbb{B}_0 = [1, 2), \qquad \mathbb{B}_1 = [2, \infty],$$

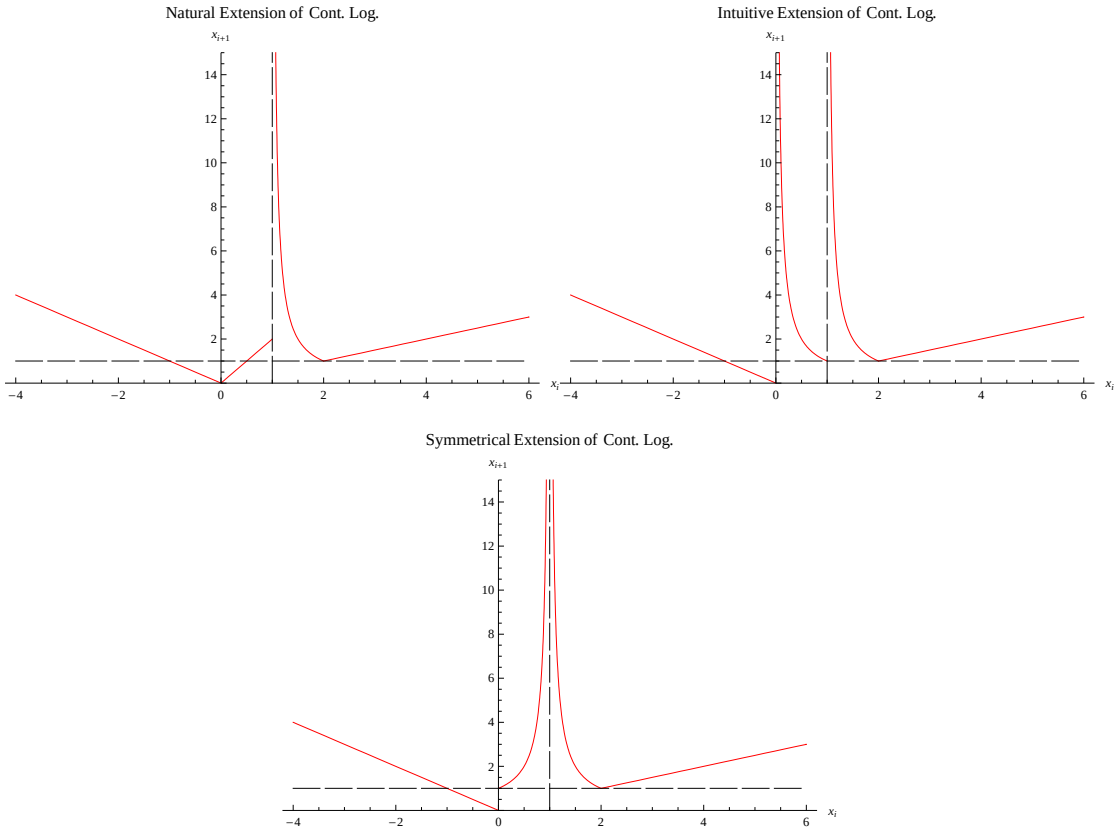

Figure 4.1: Plots of the recursive relation of considered extensions of binary continued logarithm.

and

$$\beta_- \cong \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \beta_0 \cong \begin{pmatrix} 0 & 1 \\ 1 & -1 \end{pmatrix}, \quad \beta_1 \cong \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix},$$

$$\beta_/ \cong \begin{cases} \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} & \textit{natural extension}, \\ \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} & \textit{intuitive extension}, \\ \begin{pmatrix} 0 & 1 \\ -1 & 1 \end{pmatrix} & \textit{symmetrical extension}. \end{cases}$$

The sequence $\{b_i\}$ *may terminate for* $i = n$ *such that* $x_{n+1} = \infty$.

All the three extensions use identical transformation on a domain of negative numbers and differ in a way of treating fractional values between zero and unity. The former two extensions are very similar, except their interpretation that makes the intuitive extension more beneficial—we will discuss this issue later. The symmetrical extension is mentioned mostly due to its graphical symmetry, but it otherwise does not seem to have any advantages over the former two possibilities.

We should remark on the natural extension as it was inspired by Gosper [Gosper, 1978]: *"For negative numbers we could have either '−' for 'I negated myself' or '+' for 'I incremented myself'. For fractions initially < 1, '*' would mean 'I doubled myself', the opposite of '1'."* Notice that unlike Gosper we intentionally exclude '+' as a useful transformation, because it would generate unacceptably long digit sequences for huge negative values. Instead of '∗', we use the symbol of '/'.

Tab. 4.1 gives an overview of input/output BLFT transformations with respect to the two new digits introduced by the extended representation. As shown, all the arithmetic operations needed can be realized via shift-and-add operations as a one of our requirements of a "successful"

Table 4.1: Possible rational extensions of binary continued logarithms.

	b_i	x-absorption	y-absorption	Emission
Natural	$-$	$\begin{pmatrix} -a & b & -c & d \\ -e & f & -g & h \end{pmatrix}$	$\begin{pmatrix} -a & -b & c & d \\ -e & -f & g & h \end{pmatrix}$	$\begin{pmatrix} -a & -b & -c & -d \\ e & f & g & h \end{pmatrix}$
	/	$\begin{pmatrix} a & 2b & c & 2d \\ e & 2f & g & 2h \end{pmatrix}$	$\begin{pmatrix} a & b & 2c & 2d \\ e & f & 2g & 2h \end{pmatrix}$	$\begin{pmatrix} 2a & 2b & 2c & 2d \\ e & f & g & h \end{pmatrix}$
Intuitive	$-$	$\begin{pmatrix} -a & b & -c & d \\ -e & f & -g & h \end{pmatrix}$	$\begin{pmatrix} -a & -b & c & d \\ -e & -f & g & h \end{pmatrix}$	$\begin{pmatrix} -a & -b & -c & -d \\ e & f & g & h \end{pmatrix}$
	/	$\begin{pmatrix} b & a & d & c \\ f & e & h & g \end{pmatrix}$	$\begin{pmatrix} c & d & a & b \\ g & h & e & f \end{pmatrix}$	$\begin{pmatrix} e & f & g & h \\ a & b & c & d \end{pmatrix}$
Symmetr.	$-$	$\begin{pmatrix} -a & b & -c & d \\ -e & f & -g & h \end{pmatrix}$	$\begin{pmatrix} -a & -b & c & d \\ -e & -f & g & h \end{pmatrix}$	$\begin{pmatrix} -a & -b & -c & -d \\ e & f & g & h \end{pmatrix}$
	/	$\begin{pmatrix} a+b & -a & c+d & -c \\ e+f & -e & g+h & -g \end{pmatrix}$	$\begin{pmatrix} a+c & b+d & -a & -b \\ e+g & f+h & -e & -f \end{pmatrix}$	$\begin{pmatrix} e & f & g & h \\ e-a & f-b & g-c & h-d \end{pmatrix}$

extension. All the extensions can also be shown to meet the other requirements—monotonicity, sign character of '$-$' and '/' digits, etc. It would thus seem, they are all equally useful. However, there is a difference among them with respect to applying the extensions to canonical form of continued logarithms[8].

Both natural and intuitive extensions seem to have natural interpretation with respect to the canonical representation of Eq. (2.3)

$$x_i = 2^{k_i}\left(1 + \frac{1}{x_{i+1}}\right), \quad k_i \in \mathbb{N} \cup \{0\}.$$

The '$-$' digit stands for an additive inverse, which can be incorporated into the canonical form as a sign multiplicand $s_i \in \{-1, +1\}$. In case of natural extension, the '/' digits actually represent a negative powers of k_i, which projects in extending the set of k_i's to $\mathbb{Z}$. Since '/' may occur only at the beginning of the binary expansion, this extension really affects only k_0. In case of intuitive extension, the '/' symbol realizes a multiplicative inverse, which can be embedded into Eq. (2.3) by adding an exponent $e_i \in \{-1, +1\}$. Again, '/' may occur only at the beginning and thus affects only the recursive relation of x_0. Symmetrical extension has no natural interpretation with respect to Eq. (2.3) and we thus exclude it from further discussion in Section 4.2.

Example 17. *Let us find a representation of $-\frac{1}{\sqrt{10}}$ for the naturally extended BCL. We will get the result by computing $\phi(x) = -\frac{1}{x}$ for $x = \sqrt{10} = (1000101001010010\ldots)_{BCL}$. For this purpose, we will use a BLFT algorithm for $\phi \cong \left(\begin{smallmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{smallmatrix}\right)$ with $y = \infty$ and utilize input/output transformations presented in Tab. 3.5 and 4.1. Due to $y = \infty$, it suffices to consider only the bounding values of $\frac{n_0}{d_0}$ and $\frac{n_1}{d_1}$. The process of computation is provided in Tab. 4.2, giving the prefix of representation $(-//01\ldots)_{BCL}$.*

Notice that for a given LFT, representation in intuitive EBCL can be written directly as $(-/100010\ldots)_{BCL}$; that is the represnetation of x preceded by the prefix $(-/)_{BCL}$. □

[8]The binary continued logarithms are the preferred form for practical implementation, but the canonical form has its advantage though—it is in the theoretical investigation of continued logarithms as seen already in Section 2.4. It is thus useful to have each extension of the binary form naturally fit within the canonical form too.

Table 4.2: A process of computing $-\frac{1}{\sqrt{10}}$ in natural EBCL with the use of BLFT algorithm.

m	n	o	$\phi^{(m;n,o)}$	e_m	$\frac{n_0}{d_0}$	$\frac{n_1}{d_1}$
0	0	0	$\left(\begin{smallmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{smallmatrix}\right)$	$\emptyset$	0	-1
0	1	0	$\left(\begin{smallmatrix} 0 & -1 & 0 & 0 \\ 2 & 0 & 0 & 0 \end{smallmatrix}\right)$	$\emptyset$	0	$-\frac{1}{2}$
0	2	0	$\left(\begin{smallmatrix} -1 & 0 & 0 & 0 \\ 2 & 2 & 0 & 0 \end{smallmatrix}\right)$	$-$	$-\frac{1}{2}$	$-\frac{1}{4}$
1	2	0	$\left(\begin{smallmatrix} 1 & 0 & 0 & 0 \\ 2 & 2 & 0 & 0 \end{smallmatrix}\right)$	/	$\frac{1}{2}$	$\frac{1}{4}$
2	2	0	$\left(\begin{smallmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{smallmatrix}\right)$	$\emptyset$	1	$\frac{1}{2}$
2	3	0	$\left(\begin{smallmatrix} 1 & 1 & 0 & 0 \\ 2 & 1 & 0 & 0 \end{smallmatrix}\right)$	/	$\frac{1}{2}$	$\frac{2}{3}$
3	3	0	$\left(\begin{smallmatrix} 2 & 2 & 0 & 0 \\ 2 & 1 & 0 & 0 \end{smallmatrix}\right)$	0	1	$\frac{4}{3}$
4	3	0	$\left(\begin{smallmatrix} 2 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{smallmatrix}\right)$	$\emptyset$	∞	3
4	4	0	$\left(\begin{smallmatrix} 3 & 2 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{smallmatrix}\right)$	1	3	5
5	4	0	$\left(\begin{smallmatrix} 3 & 2 & 0 & 0 \\ 2 & 0 & 0 & 0 \end{smallmatrix}\right)$	$\emptyset$	$\frac{3}{2}$	$\frac{5}{2}$
...	...	...	...	...	...	...

4.2 Canonical Extension

The previous section identified that both natural and intuitive extensions of BCL could be generalized even for CCL. This is shown in Eq. (4.1) and (4.2), respectively; compare them with the original Eq. (2.3).

$$x_i = s_i 2^{k_i}\left(1+\frac{1}{x_{i+1}}\right), \quad k_i \in \mathbb{Z},\ s_i \in \{-1,+1\}, \tag{4.1}$$

$$x_i = \left[s_i 2^{k_i}\left(1+\frac{1}{x_{i+1}}\right)\right]^{e_i}, \quad k_i \in \mathbb{N}\cup\{0\},\ s_i, e_i \in \{-1,+1\}. \tag{4.2}$$

We already remarked that the use of s_i and e_i could be limited only to $i = 0$, but using them generally does no harm; in fact, it makes the extension more regular.

The natural extension seems a bit more "natural" due to its lack of exponent e_i, but there is a practical issue against it. Regarding representation of 0, the natural extension would use an infinite number of binary digits '/', or let $k_0 = -\infty$ for the CCL form. This seems a bit awkward, especially because of the zero's special character. The intuitive extension has considerable advantage, representing 0 as a multiplicative inverse of ∞, i.e. $0 = (/)_{\mathrm{BCL}}$. For this reason, we prefer the intuitive extension and exclude the natural extension from further consideration.

We thus finally arrive at the formal definition of extended CCL representation. Notice that we used identities $s_i = \frac{1}{s_i}$ and $e_i^2 = 1$ to get the variables s_i and e_i to the other side of Eq. (4.2).

Definition 18. Extended canonical continued logarithm *representation (ECCL) of a real number $x \in \mathbb{R}$ is a sequence $\{(s_i, k_i, e_i)\}$, $s_i, e_i \in \{-1,+1\}$, $k_i \in \mathbb{N}\cup\{0\}$, $i = 0, 1, \ldots$, such that there is a sequence $\{x_i\}$, $x_{i>0} \in \mathbb{R}^* \setminus \{0\}$ and $x_0 = x$, satisfying*

$$2^{k_i} \le (s_i x_i)^{e_i} = 2^{k_i}\left(1+\frac{1}{x_{i+1}}\right) < 2^{k_i+1}. \tag{4.3}$$

The sequence $\{(s_i, k_i, e_i)\}$ terminates for $i = n$ such that $x_{n+1} = \infty$.

Example 19. *Let us find the CCL representation of $x = -\frac{1}{\sqrt{10}}$. Letting $x_0 = x$ and using the procedure below, we would get the result of*

$$x = ((-1,1,-1),\ (1,0,1),\ (1,0,1),\ (1,1,1),\ (1,1,1),\ (1,0,1),\ (1,1,1),\ldots)_{ECCL}\ .$$

The evaluation would go as follows:

$$x_i \quad \rightarrow \quad \begin{aligned} s_i &= \begin{cases} \operatorname{sgn}(x_i) & \textit{for } x_i \neq 0, \\ +1 & \textit{otherwise} \end{cases} \\ e_i &= \begin{cases} \operatorname{sgn}(\log_2 |x_i|) & \textit{for } |x_i| \neq 1, \\ +1 & \textit{otherwise} \end{cases} \\ k_i &= \lfloor \log_2 s_i x_i^{e_i} \rfloor \end{aligned} \quad \rightarrow \quad x_{i+1} = \frac{2^{k_i}}{s_i x_i^{e_i} - 2^{k_i}} \ .$$

□

4.3 Revision of Arithmetic

Principles of arithmetic discussed in Section 3 allowed us to compute mappings of $\mathbb{I} \times \mathbb{I} \rightarrow \mathbb{I}$. Considering presented extensions, we could compute mappings of $\mathbb{I} \times \mathbb{I} \rightarrow \mathbb{R}^*$. Extension of input domains from $\mathbb{I}$ to $\mathbb{R}^*$ requires revising some of the principles we used so far.

Lemma 20. *Suppose that $\{b_i\}$ is an (intuitive) EBCL representation of a real number x and that $\{x_i\}$ is a corresponding sequence of continuations. Then for any $j \geq 2$ it is $x_j \in \mathbb{I} = [1, \infty]$.*

Proof. By inspection of all possible cases in interval of $(\infty, 1)$, one would find that $(-)_{\mathrm{BCL}}$, $(/)_{\mathrm{BCL}}$ and $(-/)_{\mathrm{BCL}}$ are the only prefixes that correspond to values outside $\mathbb{I}$. □

Lemma 21. *Suppose that $\{(s_i, k_i, e_i)\}$ is an ECCL representation of a real number x and that $\{x_i\}$ is a corresponding sequence of continuations. Then for any $j \geq 1$ it is $x_j \in \mathbb{I} = [1, \infty]$.*

Proof. Since by Eq. (4.3) it must be $2^{k_i} \leq 2^{k_i}\left(1 + \frac{1}{x_{i+1}}\right) < 2^{k_i+1}$, we have that $x_{i+1} \in (1, \infty]$ for any i. □

The lemmas guide us on how the assumption of range of an input argument changes with processing of its elements. E.g. for EBCL, we have back the assumption the input argument lies within $\mathbb{I}$ no later than after processing its first two input arguments. Thus the `range` function should be adapted to return $\{\infty\}$ for a terminated argument, $\mathbb{I}$ for an argument with two or more absorbed elements, and $\mathbb{R}^*$ otherwise[9].

Decision process is affected appropriately. Originally, we assumed that an argument's domain is a subset of $\mathbb{I} \times \mathbb{I}$ and so the decision may execute only if this condition is satisfied. The well-definedness condition (3.5) still holds, but we may add an extra condition to be sure that numerator of the computed (B)LFT does not change its sign over the assumed domain either; use of this extra condition depends on how values of $\frac{n_i}{d_i}$ are evaluated. For instance, the decision process in Alg. 3 uses simple comparisons to determine if some of the decision conditions hold; having the numerator change its sign might result in a false indication of decision interval. In this particular case, we also assume that the denominator's coefficients are always kept positive, thus any sign change happening only within the numerator.

Example 22. *Compute a BLFT $\phi = x + y$, $x = -\sqrt{10}$, $y = 1/\sqrt{10}$, using Alg. 2 and the modifications above. Representation of the arguments in (intuitive) EBCL comes immediately from the representation of $\sqrt{10}$; thus $x = (-10001010\ldots)_{BCL}$ and $y = (/10001010\ldots)_{BCL}$. Computation follows Alg. 2, but uses the simplified* `range` *estimation of input domains and Alg. 3 to implement* `decide` *procedure; its progress is given in Tab. 4.3. The computed result is $\phi(x, y) = (-1010110\ldots)_{BCL} \in [2.8, 2.89)$.* □

[9] We may improve this simple implementation by returning $\mathbb{I}$ whenever we detect either '0' or '1' digit at the input.

Algorithm 3: Decision process `decide(`ϕ`,`$\mathbb{D}$`)` for EBCL representation.

Input: ϕ – a BLFT given as an eight-tuple of its coefficients.
Input: $\mathbb{D}$ – a domain in $\mathbb{R}^* \times \mathbb{R}^*$ over which to consider ϕ.
Output: A digit potential for emission, if it exists, or $\emptyset$ otherwise.

```
1  c = ∅;
2  if (𝔻 ⊈ 𝕀 × 𝕀) then
3  |   return c;
4  end
5  for i=0 to 3 do
6  |   n_i = numerator(i, 𝔻);
7  |   d_i = denominator(i, 𝔻);
8  end
9  if (φ is well-defined on 𝔻) then                          // see Eq.(3.5)
10 |   if (φ changes sign on 𝔻) then                         // no way to emit
11 |   |   c = ∅;
12 |   else if (sgn(n_0) == −sgn(d_0)) then                  // φ(𝔻) ⊆ [∞, 0)
13 |   |   c = −;
14 |   else if (n_i < d_i for ∀i) then                       // φ(𝔻) ⊆ [0, 1)
15 |   |   c = /;
16 |   else if (d_i ≤ n_i < 2d_i for ∀i) then                // φ(𝔻) ⊆ [1, 2)
17 |   |   c = 0;
18 |   else if (2d_i ≤ n_i for ∀i) then                      // φ(𝔻) ⊆ [2, ∞]
19 |   |   c = 1;
20 |   end
21 end
22 return c;
```

Table 4.3: A snapshot of variables and parameters taken at line 5 of Alg. 2 computed for EBCL and $\phi = x + y$ with $x = -\sqrt{10}$, $y = 1/\sqrt{10}$.

m	n	o	$\phi^{(m;n,o)}$	e_m	$\frac{n_0}{d_0}$	$\frac{n_1}{d_1}$	$\frac{n_2}{d_2}$	$\frac{n_3}{d_3}$
0	0	0	$\left(\begin{smallmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{smallmatrix}\right)$	$\emptyset$	n/a	∞	∞	2
0	1	0	$\left(\begin{smallmatrix} 0 & 1 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{smallmatrix}\right)$	$\emptyset$	n/a	∞	∞	n/a
0	1	1	$\left(\begin{smallmatrix} -1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{smallmatrix}\right)$	$\emptyset$	∞	-1	∞	0
0	2	1	$\left(\begin{smallmatrix} -2 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{smallmatrix}\right)$	$\emptyset$	∞	-2	∞	-1
0	2	2	$\left(\begin{smallmatrix} -4 & 0 & 0 & 1 \\ 0 & 2 & 0 & 0 \end{smallmatrix}\right)$	$\emptyset$	∞	-2	∞	$-\frac{3}{2}$
0	3	2	$\left(\begin{smallmatrix} -4 & -4 & 1 & 0 \\ 2 & 0 & 0 & 0 \end{smallmatrix}\right)$	$-$	-2	-4	$-\frac{3}{2}$	$-\frac{7}{2}$
1	3	2	$\left(\begin{smallmatrix} 4 & 4 & -1 & 0 \\ 2 & 0 & 0 & 0 \end{smallmatrix}\right)$	$\emptyset$	2	4	$\frac{3}{2}$	$\frac{7}{2}$
1	3	3	$\left(\begin{smallmatrix} 3 & 4 & 4 & 4 \\ 2 & 0 & 2 & 0 \end{smallmatrix}\right)$	$\emptyset$	$\frac{3}{2}$	$\frac{7}{2}$	$\frac{7}{4}$	$\frac{15}{4}$
1	4	3	$\left(\begin{smallmatrix} 7 & 3 & 8 & 4 \\ 2 & 2 & 2 & 2 \end{smallmatrix}\right)$	1	$\frac{7}{2}$	5	$\frac{15}{4}$	$\frac{11}{4}$
2	4	3	$\left(\begin{smallmatrix} 7 & 3 & 8 & 4 \\ 4 & 4 & 4 & 4 \end{smallmatrix}\right)$	0	$\frac{7}{4}$	$\frac{10}{8}$	$\frac{15}{8}$	$\frac{11}{8}$
3	4	3	$\left(\begin{smallmatrix} 4 & 4 & 4 & 4 \\ 3 & -1 & 4 & 0 \end{smallmatrix}\right)$	$\emptyset$	$\frac{4}{3}$	4	$\frac{8}{7}$	$\frac{8}{3}$
3	4	4	$\left(\begin{smallmatrix} 8 & 8 & 4 & 4 \\ 7 & -1 & 3 & -1 \end{smallmatrix}\right)$	$\emptyset$	$\frac{8}{7}$	$\frac{8}{3}$	$\frac{6}{5}$	3
3	5	4	$\left(\begin{smallmatrix} 16 & 8 & 8 & 4 \\ 6 & 7 & 2 & 3 \end{smallmatrix}\right)$	$\emptyset$	$\frac{8}{3}$	$\frac{24}{13}$	3	2
3	5	5	$\left(\begin{smallmatrix} 24 & 12 & 16 & 8 \\ 8 & 10 & 6 & 7 \end{smallmatrix}\right)$	$\emptyset$	3	2	$\frac{20}{7}$	$\frac{60}{31}$
3	6	5	$\left(\begin{smallmatrix} 48 & 12 & 32 & 8 \\ 16 & 10 & 12 & 7 \end{smallmatrix}\right)$	1	3	$\frac{30}{13}$	$\frac{20}{7}$	$\frac{20}{9}$
4	6	5	$\left(\begin{smallmatrix} 24 & 6 & 16 & 4 \\ 16 & 10 & 12 & 7 \end{smallmatrix}\right)$	0	$\frac{3}{2}$	$\frac{15}{13}$	$\frac{10}{7}$	$\frac{10}{9}$
5	6	5	$\left(\begin{smallmatrix} 16 & 10 & 12 & 7 \\ 8 & -4 & 4 & -3 \end{smallmatrix}\right)$	1	2	$\frac{13}{2}$	$\frac{7}{3}$	9
6	6	5	$\left(\begin{smallmatrix} 16 & 10 & 12 & 7 \\ 16 & -8 & 8 & -6 \end{smallmatrix}\right)$	$\emptyset$	1	$\frac{13}{4}$	$\frac{7}{6}$	$\frac{9}{2}$
6	6	6	$\left(\begin{smallmatrix} 32 & 20 & 12 & 7 \\ 32 & -16 & 8 & -6 \end{smallmatrix}\right)$	$\emptyset$	1	$\frac{13}{4}$	$\frac{11}{10}$	$\frac{71}{18}$
6	7	6	$\left(\begin{smallmatrix} 52 & 32 & 19 & 12 \\ 16 & 32 & 2 & 8 \end{smallmatrix}\right)$	$\emptyset$	$\frac{13}{4}$	$\frac{21}{12}$	$\frac{71}{18}$	$\frac{115}{58}$
6	7	7	$\left(\begin{smallmatrix} 71 & 44 & 52 & 32 \\ 18 & 40 & 16 & 32 \end{smallmatrix}\right)$	$\emptyset$	$\frac{71}{18}$	$\frac{115}{58}$	$\frac{133}{34}$	$\frac{199}{106}$
6	8	7	$\left(\begin{smallmatrix} 71 & 22 & 52 & 16 \\ 18 & 20 & 16 & 16 \end{smallmatrix}\right)$	1	$\frac{71}{18}$	$\frac{93}{38}$	$\frac{133}{34}$	$\frac{171}{70}$
7	8	7	$\left(\begin{smallmatrix} 71 & 22 & 52 & 16 \\ 36 & 40 & 32 & 32 \end{smallmatrix}\right)$	0	$\frac{71}{36}$	$\frac{93}{76}$	$\frac{133}{68}$	$\frac{171}{140}$
...	...	...	...	...	...	...	...	...

Table 4.4: Snapshot of parameters observed for $\phi = xy = \sqrt{2} \cdot \sqrt{2}$. n_i/d_i are ϕ's values at the corners of $\mathbb{I} \times \mathbb{I}$, a, e are coefficients of ϕ.

Parameter	$\phi^{(0;0,0)}$	$\phi^{(0;5,5)}$	$\phi^{(0;20,20)}$	$\phi^{(0;40,40)}$
n_0/d_0	1.0	2.25	1.9999649	2.000000000153
n_1/d_1	2.0	2.10	1.9999855	2.0
n_2/d_2	2.0	2.10	1.9999855	2.0
n_3/d_3	4.0	1.96	2.0000060	1.999999999847
a	1	36	228 484	$130\ 515 \cdot 10^5$
e	1	16	114 244	$65\ 257 \cdot 10^5$

4.4 Computability Issues

Continued logarithm arithmetic introduced in this section can compute any rational transformation of any rational arguments. This comes form the fact that rational arguments must terminate at some time, and so must their result.

The arithmetic may also compute rational transformations of real arguments, as shown in previous examples, but there are cases when such arithmetic fails. One such example is computing $\phi = xy$ for $x = y = \sqrt{2}$. This particular example is illustrated in Tab. 4.4, which shows some of the ϕ's parameters. Namely, the table lists fractions $\frac{n_i}{d_i}$ as defined by Eq. (3.4). It also provides values of fractional coefficients a and e. As the figures illustrate, the range of $\phi(\mathbb{I} \times \mathbb{I})$ spreads in neighborhood of 2, but none of the decision conditions is fully satisfied and does not seem to ever be.

This behavior is easy to explain by considering the discrete character of the representation; we consider EBCL here. The result of $\sqrt{2} \cdot \sqrt{2}$ is 2, which is actually one of the decision borders, at which the character of the representation changes (see e.g. Fig. 4.1). If both arguments were $x, y \geq \sqrt{2}$, then `decide` would indicate to emit 1; but if some argument was less than $\sqrt{2}$, then 0 would be emitted. However, since both arguments are infinite, the evaluation process will never discover that none of the arguments is less than $\sqrt{2}$ and thus it will never emit a single digit of the result.

5 Real Extension

To address the computability issues discussed in Section 4.4, most of existing lazy sequence, real number representations have to be made redundant. This fact is well-known and evidenced in the literature, e.g. [Wiedmer, 1980, Vuillemin, 1990, Niqui, 2007, Kornerup & Matula, 1990]. In this chapter, we present such extension for continued logarithms. This time, we start with the extension of the canonical representation first and then project the results into the binary form.

5.1 Canonical Extension

Lazy sequence representations share the common property that their prefixes represent sets of numbers, range of which is determined by all possible suffixes. The fundamental idea of redundancy is to allow such prefixes represent numbers both above and below the value determined by each particular prefix itself. This is indeed the idea behind signed-digit positional number representation.

In case of continued logarithms, this principle could be well utilized. Take e.g. $x = 3$, for which the following equality holds:

$$3 = 2^1 + \frac{2^1}{2^1} = 2^2 - \frac{2^2}{2^2}.$$

That is, we can use the negative values of continuations x_i to decrease the value of x_{i-1}, if it was set too high. This idea has been discussed in detail in [Brabec, 2008] and herein we provide its brief summary.

Definition 23. Extended redundant canonical continued logarithm *representation (ERCCL) of a real number* $x \in \mathbb{R}$ *is a sequence of triples* $\{(s_i, k_i, e_i)\}$, $s_i, e_i \in \{-1, 1\}$, $k_i \in \mathbb{N} \cup \{0\}$, $i = 0, 1, \ldots$, *such that there exists a sequence* $\{x_i\}$, $x_{i>0} \in \mathbb{R}^* \setminus \{0\}$ *and* $x_0 = x$, *with*

$$2^{k_i - 1} < (s_i x_i)^{e_i} = 2^{k_i} \left(1 + \frac{1}{x_{i+1}}\right) < 2^{k_i + 1}. \tag{5.1}$$

The sequence $\{(s_i, k_i, e_i)\}$ *terminates for* $i = n$ *such that* $x_{n+1} = \infty$.

Almost equal to Def. 18, the slight difference is in the lower bound of Eq. (5.1), which allows every continuation x_i to choose for k_i either $\lfloor \log_2 s_i x_i^{e_i} \rfloor$ or $\lceil \log_2 s_i x_i^{e_i} \rceil$. The two choices collide into one if $s_i x_i^{e_i}$ is a power of 2.

Example 24. *Find ERCCL representations of* $x = 5$. *The procedure of computing an ERCCL representation follows that of Example 19, but it must additionally accommodate for several choices of* (s_i, k_i, e_i). *The adapted procedure is shown below, where the* `select` *function choose arbitrarily between the two possibilities:*

$$s_i = \begin{cases} \operatorname{sgn}(x_i) & \textit{for } x_i \neq 0, \\ +1 & \textit{otherwise} \end{cases}$$

$$x_i \quad \rightarrow \quad e_i = \begin{cases} -1 & \textit{if } |x_i| \in [0, \frac{1}{2}], \\ \textit{select}(-1, +1) & \textit{if } |x_i| \in (\frac{1}{2}, 2), \\ +1 & \textit{otherwise} \end{cases} \quad \rightarrow \quad x_{i+1} = \frac{2^{k_i}}{s_i x_i^{e_i} - 2^{k_i}} \ .$$

$$0 \leq \quad k_i = \textit{select}(\lceil \log_2 s_i x_i^{e_i} \rceil, \lfloor \log_2 s_i x_i^{e_i} \rfloor)$$

Tab. 5.1 demonstrates the use of this procedure, assuming that `select` *always choses the first of its operands. Other representations are below:*

$$\begin{aligned}
&(\ (1,2,1),\ (1,2,1)\ )_{ERCCL}\ , \\
&(\ (1,3,1),\ (-1,1,1)\ (1,1,1),\ (1,1,1)\ )_{ERCCL}\ , \\
&(\ (1,3,1),\ (-1,1,1)\ (1,2,1),\ (-1,2,1)\ )_{ERCCL}\ , \\
&(\ (1,3,1),\ (-1,2,1)\ (-1,1,1),\ (1,1,1)\ )_{ERCCL}\ , \\
&(\ (1,3,1),\ (-1,2,1)\ (-1,2,1),\ (-1,2,1)\ )_{ERCCL}\ .
\end{aligned}$$

Table 5.1: A sample process of finding an ERCCL representation of $x = 5$.

i	0	1	2	3	4
x_i	5	$-\frac{8}{3}$	-3	-4	∞
s_i	$+1$	-1	-1	-1	
k_i	3	2	2	2	
e_i	$+1$	$+1$	$+1$	$+1$	

□

It can be shown that the ERCCL representation possesses all the necessary properties. In particular, the contractivity and interval openness. The contractivity ensures that the representation converges to a value being represented as the length of its prefix increases. The interval openness makes each prefix represent values both above and below its nominal value.

Combination of these properties is called *interior containment property* [Boehm et al., 1986] and acts as a necessary condition for real number computability. The following theorem covers the contractivity property.

Theorem 25. *Let us have an ERCCL representation* $\{(s_i, k_i, e_i)\}$, $i = 0, 1, \ldots,$ *of a real number* x. *If we define a sequence of functions* $\{\gamma_j\}$, $j = 1, 2, \ldots,$ *such that*

$$\gamma_j(\chi) = \left(s_0 2^{k_0} + \frac{s_0 2^{k_0}}{\left(\ddots \left(s_{j-1} 2^{k_{j-1}} + \frac{s_{j-1} 2^{k_{j-1}}}{\chi} \right)^{e_{j-1}} \ldots \right)^{e_1}} \right)^{e_0}, \qquad \chi \in \mathbb{R}^* \setminus \{0\},$$

then the range of these functions contracts with increasing j *so that*

$$\gamma_1(\mathbb{I}_1) \supsetneq \gamma_2(\mathbb{I}_2) \supsetneq \ldots \supsetneq \{x\},$$

where $\mathbb{I}_j$ *is defined solely by the value of* $(s_{j-1}, k_{j-1}, e_{j-1})$ *as follows*

$$\mathbb{I}_j = \begin{cases} (\infty, -2) & \text{if } (s_{j-1}, k_{j-1}, e_{j-1}) = (1, 0, -1), \\ (1, \infty) & \text{if } (s_{j-1}, k_{j-1}, e_{j-1}) \in \{(-1, 1, 1), (1, 0, 1)\}, \\ [\infty, -2) \cup (1, \infty] & \text{otherwise.} \end{cases}$$

The only difference between finite and infinite ERCCL representation is that for the finite representation of length $n + 1$ *we have* $x = \gamma_{n+1}(\{\infty\})$.

Proof. Refer to [Brabec, 2008]. □

Example 26. *Let us show the contractivity for* $x = 5$ *and its representation*

$$((1, 3, 1),\ (-1, 2, 1)\ (-1, 1, 1),\ (1, 1, 1))_{ERCCL}\ .$$

The results are given in Tab. 5.2. Except for ranges of $\gamma_j(\mathbb{I}_j)$, *we also provide figures of* $\gamma_j(\mathbb{I})$, *where* $\mathbb{I} = [\infty, -2) \cup (1, \infty]$. *The interval* $\mathbb{I}$ *actually represents the base interval of the redundant representation [Brabec, 2008] so that* $x_j \in \mathbb{I}$ *for* $j > 0$. *Unfortunately, as we evidence in the table, the statement* $\gamma_j(\mathbb{I}) \supsetneq \gamma_{j+1}(\mathbb{I})$ *does not hold.*

This is particularly the case of the three values of $(s_j, k_j, e_j) \in \{(1, 0, 1), (-1, 1, 1), (1, 0, -1)\}$ *representing continuations at the border of that base interval. For these three cases it happens that* $\left[s_j 2^{k_j} \left(1 + \frac{1}{\mathbb{I}}\right)\right]^{e_j} \nsubseteq \mathbb{I}$. *However, as analysis in [Brabec, 2008] shows, the base interval in these cases may be more tightly restricted to the form of* $\mathbb{I}_{j+1}$ *used in Theorem 25.*

Table 5.2: Example of ERCCL contractivity.

j	$(s_{j-1}, k_{j-1}, e_{j-1})$	γ_j	$\gamma_j(\mathbb{I}_j)$	$\gamma_j(\mathbb{I})$
1	$(1, 3, 1)$	$\left(\begin{smallmatrix} 8 & 8 \\ 1 & 0 \end{smallmatrix}\right)$	$(4, 16)$	$(4, 16)$
2	$(-1, 2, 1)$	$\left(\begin{smallmatrix} 6 & 8 \\ 1 & 1 \end{smallmatrix}\right)$	$(4, 7)$	$(4, 7)$
3	$(-1, 1, 1)$	$\left(\begin{smallmatrix} 4 & 12 \\ 1 & 2 \end{smallmatrix}\right)$	$(4, \frac{16}{3})$	$(\infty, \frac{16}{3})$
4	$(1, 1, 1)$	$\left(\begin{smallmatrix} 10 & 4 \\ 2 & 1 \end{smallmatrix}\right)$	$(\frac{14}{3}, \frac{16}{3})$	$(\frac{14}{3}, \frac{16}{3})$

□

5.2 Binary Extension

Using the correspondence between canonical and binary form, one could find, having an ERCCL representation, a corresponding binary representation. However, finding a redundant binary form directly is less obvious and its formal definition lacks the elegance of the canonical form.

Definition 27. Extended redundant binary continued logarithm *representation (ERBCL) of a real number* $x \in \mathbb{R}$ *is a sequence of* digits $\{b_i\}$, $b_i \in \{0, 1, /, -\}$, $i = 0, 1, \ldots$, *such that there is a sequence* $\{x_i\}$, $x_i \in \mathbb{R}^*$ *and* $x_0 = x$, *satisfying*

$$x_{i+1} = \beta_{b_i}(x_i) \qquad \textit{and} \qquad b_i = \begin{cases} 1 & \textit{if } x_i \in \mathbb{B}_1 = [\infty, -2) \cup (1, \infty], \\ 0 & \textit{if } x_i \in \mathbb{B}_0 = (\frac{1}{2}, 2), \\ / & \textit{if } x_i \in \mathbb{B}_/ = (-1, 1) \textit{ and Eq. (5.2)}, \\ - & \textit{if } x_i \in \mathbb{B}_- = (\infty, 0), \end{cases}$$

where

$$\beta_- \cong \left(\begin{smallmatrix} -1 & 0 \\ 0 & 1 \end{smallmatrix}\right), \quad \beta_0 \cong \left(\begin{smallmatrix} 0 & 1 \\ 1 & -1 \end{smallmatrix}\right), \quad \beta_1 \cong \left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right), \quad \beta_/ \cong \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right).$$

The sequence $\{b_i\}$ *may terminate for* $i = n$ *such that* $x_{n+1} = \infty$. *The digit* / *may be used only at the beginning of the expansion as defined by the following condition:*

$$i = 0 \ \vee \ (i = 1 \ \wedge \ b_0 = -). \tag{5.2}$$

When compared to a non-redundant representation of Def. 16, one can notice the major difference being in decision intervals $\mathbb{B}_{b_i}$. Except $\mathbb{B}_-$, all intervals have been extended so that the neighboring intervals overlap. Thanks to those overlaps, there are no strict decision borders and one may choose between two alternatives how to develop yet unexpanded continuation x_i.

A special remark should be given on use of the '/' digit. Non-redundant representation implicitly ensured that '/' might be used only at the beginning of the expansion as, once $[0, 1)$ had been transformed to $(1, \infty]$, there arose no need for its further use. With redundant representation and interval overlaps, it may happen that a number in $(1, 2)$ is transformed to $(\frac{1}{2}, 1)$ that would otherwise give an equal chance to use '0' as well as '/'. Since the latter is not desirable, we explicitly disallowed such possibility in Def. 27 by condition (5.2).

Lemma 28. *Let* $\{b_i\}$ *be an ERBCL representation with* $\{k_j\}$, $j = 0, 1, \ldots$, *being an ordered sequence of indices of all '0' digits. Consider an arbitrary* $m \geq 0$ *such that* k_m *is defined and is not the last element of* $\{k_j\}$. *Then:*

(a) If $\exists p$, $k_m < p < k_{m+1}$, *then* $b_p \in \{1, -\}$.

(b) If $\exists p, q$, $k_m < p, q < k_{m+1}$ *and* $b_p = b_q = -$, *then* $p = q$.

(c) If $\exists p$, $k_m < p < k_{m+1}$ *such that* $b_p = -$, *then* $k_{m+1} - k_m - 1 > 1$.

Proof. (a) Since '/' may occur only at the beginning of expansion, digits '1' and '−' are the only one that may appear between '0's.

(b) Assume that there are p, q, $p < q$, such that $b_p = b_q = -$. It is thus that $x_p, x_q \in [\infty, 0)$ and

$$x_q \in \prod_{p+1}^{q-1} \left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} -1 & 0 \\ 0 & 1 \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} \mathbb{B}_p \\ 1 \end{smallmatrix}\right) = \left(\begin{smallmatrix} -2^{q-p-1} & 0 \\ 0 & 1 \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} [\infty, 0) \\ 1 \end{smallmatrix}\right) = (0, \infty],$$

which is, however, a contradiction.

(c) This part of the lemma actually says that if '$-$' occurs between two '0's, then a '1' digit has to occur there too. Since $b_{k_m} = 0$, then $x_{k_m+1} \in [\infty, -2) \cup (1, \infty]$. However, since $b_p = -$, then also $x_{k_m+1} \in [\infty, 0)$. That is, $x_{k_m+1} \in [\infty, -2)$ and the result immediately follows.

□

Theorem 29. *If $\{b_i\}$ is an ERBCL representation of a real number $x \in \mathbb{R}$, there is a an equivalent sequence $\{(s_j, k_j, e_j)\}$ satisfying the defining condition Eq. (5.1) of ERCCL representation.*

Proof. We will show how, for the given $\{b_i\}$, to find the corresponding sequence $\{(s_j, k_j, e_j)\}$ such that it satisfies Eq. (5.1).

Let us define a sequence $\{n_j\}$, $j = 0, 1, \ldots$, such that:

1. $n_0 < n_1 < \ldots$, and
2. $\forall j: \ b_{n_j} = 0$, and
3. $\forall j$: there is no m, $n_j < m < n_{j+1}$, such that $b_m = 0$.

In other words, $\{n_j\}$ is the ordered sequence of indices of all zero bits within $\{b_i\}$.

Let us then define sequences $\{e'_j\}$ and $\{s'_j\}$ assuming $n_{-1} = -1$ as follows:

$$e'_j = \begin{cases} 1 & \text{if } j = 0 \text{ and } \exists i < n_0, \ b_i = /, \\ 0 & \text{otherwise.} \end{cases}$$

$$s'_j = \begin{cases} 1 & \text{if } \exists i, \ n_{j-1} < i < n_j, \ b_i = -, \\ 0 & \text{otherwise.} \end{cases}$$

That is, sequences of e'_j and s'_j are "bit masks" that indicate whether '/' and '$-$' digits, respectively, occur between $(j-1)$-th and j-th '0' bit. Using these sequences, we may define triples (s_j, k_j, e_j) as follows:

$$s_j = (-1)^{s'_j},$$
$$e_j = (-1)^{e'_j},$$
$$k_j = n_j - n_{j-1} - s'_j - e'_j - 1.$$

The effect of these assignments is that $s_j \in \{-1, +1\}$ corresponds to a sign of j-th sub-expansion $\{b_i\}_{i=n_{j-1}+1}^{n_j}$, while $k_j \in \mathbb{N} \cup \{0\}$ counts the number of '1's in the same sub-expansion. Finally, $e_j \in \{-1, +1\}$ specifies whether the value of j-th sub-expansion is inverted or not; this is indeed only the question of $j = 0$ as $e_j = 1$ for $j > 0$.

Now, we have to prove that the sequence of triples (s_j, k_j, e_j) found above satisfies Eq. (5.1). For $j \geq 0$ and $n_j < m < n_{j+1}$ being an index of '$-$' digit, if it exists, it is:

$$\begin{aligned} x_{n_j+1} &\cong \prod_{n_j+1}^{m-1} \left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} (-1)^{s'_{j+1}} & 0 \\ 0 & 1 \end{smallmatrix}\right) \cdot \prod_{m+s'_{j+1}}^{n_{j+1}-1} \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} x_{n_{j+1}+1} \\ 1 \end{smallmatrix}\right) \\ &= (-1)^{s'_{j+1}} 2^{n_{j+1}-n_j-s'_{j+1}-1} \left(1 + \frac{1}{x_{n_{j+1}+1}}\right) \\ &= s_{j+1} 2^{k_{j+1}} \left(1 + \frac{1}{x_{n_{j+1}+1}}\right) = \left[s_{j+1} 2^{k_{j+1}} \left(1 + \frac{1}{x_{n_{j+1}+1}}\right)\right]^{e_{j+1}}. \end{aligned}$$

For n_0, examining all possible combinations of s_0' and e_0' would also prove that

$$x_0 = \left[s_0 2^{k_0} \left(1 + \frac{1}{x_{n_0+1}} \right) \right]^{e_0}.$$

Still assuming that $n_{-1} = -1$, we could combine the two previous equations into the following one for $j \geq 0$:

$$x_{n_{j-1}+1} = \left[s_j 2^{k_j} \left(1 + \frac{1}{x_{n_j+1}} \right) \right]^{e_j}.$$

By a general equivalence $s_j = \frac{1}{s_j}$ and $e_j^2 = 1$, we transform the previous result into:

$$s_j x_{n_{j-1}+1}^{e_j} = 2^{k_j} \left(1 + \frac{1}{x_{n_j+1}} \right).$$

Finally, as $b_{n_j} = 0$ implies that $x_{n_j} \in \mathbb{B}_0 = (\frac{1}{2}, 2)$, it is $x_{n_j+1} \in \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)^* \cdot \left(\begin{smallmatrix} \mathbb{B}_0 \\ 1 \end{smallmatrix}\right) \subseteq [\infty, -2) \cup (1, \infty]$ and we thus get

$$2^{k_j - 1} < s_j x_{n_{j-1}+1}^{e_j} = 2^{k_j} \left(1 + \frac{1}{x_{n_j+1}} \right) < 2^{k_j+1}.$$

Now, if we define a sequence $\{y_j\}$ such that $y_j = x_{nj-1+1}$, then sequences $\{(s_j, k_j, e_j)\}$ and $\{y_j\}$ satisfy all the conditions put by Def. 23 on ERCCL representation. This completes the proof. □

Corollary 30. *Let $\{b_i\}$ be an ERBCL representation of a real number $x \in \mathbb{R}$. We define a sequence $\{\gamma_i\}$, $\gamma_i : \mathbb{R}^* \to \mathbb{R}^*$, as follows:*

$$\gamma_i(\chi) = \beta_{b_0} \circ \beta_{b_1} \circ \ldots \circ \beta_{b_i}(\chi). \tag{5.3}$$

Let $\{n_j\}$, $j = 0, 1, \ldots$, be an increasing, ordered sequence of indices of all '0' digits within $\{b_i\}$, then the sequence of intervals $\{\gamma_{n_j}(\mathbb{D}_{n_j})\}$, where $\mathbb{D}_{n_j} = \mathbb{I}_{j+1}$ for an equivalent ERCCL expansion (see Theorem 25), satisfies

$$\gamma_{n_0}(\mathbb{D}_{n_0}) \supsetneq \gamma_{n_1}(\mathbb{D}_1) \supsetneq \ldots \supsetneq \{x\}.$$

This result is extremely important as it proves that ERBCL representation has a contracting character at a coarser level of granularity, i.e.

$$\gamma_{n_i}(\mathbb{D}_{n_i}) \supsetneq \gamma_{n_{i+1}}(\mathbb{D}_{n_{i+1}}).$$

Without correspondence with canonical form, obtaining the same result would be more complicated because of ERBCL's lesser regularity. Though we would really like to see that the same result holds even at the digit-level of granularity, i.e.

$$\gamma_i(\mathbb{D}_i) \supsetneq \gamma_{i+1}(\mathbb{D}_{i+1}) \tag{5.4}$$

to hold for almost[10] all $i \geq 0$ and $\mathbb{D}_i$ appropriately chosen.

The real issue is the choice of $\mathbb{D}_i$ as a largest domain of the continuation x_{i+1} such that the two relations

$$x_i = \beta_{b_i}(x_{i+1}) \qquad \text{and} \qquad x_{i+1} = \beta_{b_{i+1}}(x_{i+2}).$$

are satisfied. This domain depends at least on the choice of b_i and b_{i+1}, but sometimes also on other, preceding elements of the corresponding ERBCL prefix. We conjecture without a proof

[10] We will see that there are j's such that the subset relation is not proper, i.e. $\gamma_{j-1}(\mathbb{D}_{j-1}) \supseteq \gamma_j(\mathbb{D}_j)$. However, the number of such j's is finite—or countably infinite for infinite continued logarithms.

that the number of such $\mathbb{D}_i$'s is limited, but it is still large enough for us here to analyze them all.

In the sequel, we prove statements that form a part of the analysis of Eq. (5.4). Following these proofs and using their principle to examine other patterns of elements that appear within ERBCL expansions, one might evidence that Eq. (5.4) really holds for almost all i.

Theorem 31. *Let $\{b_i\}$ be an ERBCL representation of a real number $x \in \mathbb{R}$. If there is an $j > 0$ such that $b_{j-1} = b_j = 0$, then $x_{j+1} \in (1, \infty]$.*

Proof. Since $b_{j-1} = 0$, then $x_{j-1} \in (\frac{1}{2}, 2)$. Furthermore, since $b_j = 0$, then $x_{j+1} \in [\infty, -2) \cup (1, \infty]$. Suppose that $x_{j+1} \in [\infty, -2)$. Because of

$$x_{j-1} \cong \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} x_{j+1} \\ 1 \end{smallmatrix}\right) \cong \frac{2x_{j+1} + 1}{x_{j+1} + 1},$$

it would be $x_{j-1} \in (2, 3)$, which is however a contradiction. □

The previous theorem showed that certain patterns of ERBCL digits impose some constraints on their continuations. E.g. b_i being '0' makes, by definition, its continuation x_{i+1} be in $[\infty, -2) \cup (1, \infty]$. However, as we saw, if b_{i-1} is '0' too, this implicit domain is tighter; $(1\infty]$ in particular.

Theorem 32. *Let $\{b_i\}$ be an ERBCL representation of a real number $x \in \mathbb{R}$ and sequence $\{\gamma_i\}$ be defined as in Corollary 30. Consider p and q such that $p < q$, $b_p = 0$ and for $j = p+1, p+2, \ldots, q$ it is $b_j = 1$. Then*

$$\gamma_p(\mathbb{D}) \supseteq \gamma_{p+1}(\mathbb{D}') \supsetneq \gamma_{p+2}(\mathbb{D}') \supsetneq \ldots \supsetneq \gamma_q(\mathbb{D}'), \tag{5.5}$$

where $\mathbb{D} = [\infty, -2) \cup (1, \infty]$ and $\mathbb{D}' = [\infty, -1) \cup (\frac{1}{2}, \infty]$.

Proof. Consider $\gamma_j \cong \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$. Then $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} \mathbb{D} \\ 1 \end{smallmatrix}\right) \subseteq \mathbb{Y}$, where either $\mathbb{Y} = \left(\frac{a+b}{c+d}, \frac{2a-b}{2c-d}\right)$ if $\frac{a+b}{c+d} \leq \frac{2a-b}{2c-d}$ or $\mathbb{Y} = \left(\frac{2a-b}{2c-d}, \frac{a+b}{c+d}\right)$ otherwise. We will consider the former case here, while the proof of the other one is analogous.

As we assume $\frac{a+b}{c+d} \leq \frac{2a-b}{2c-d}$, we get[11]

$$\begin{aligned} (a+b)(2c-d) &\leq (c+d)(2a-b) \\ 2ac + 2bc - ad - bd &\leq 2ac + 2ad - bc - bd \\ 0 &\leq 3(ad - bc) \end{aligned}$$

which is equivalent to

$$0 < ad - bc \tag{5.6}$$

as we do not need to consider $ad - bc = 0$, in which case $\gamma_p(\mathbb{D}) = \text{const}$.

We will show that it is

$$\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} 2^k & 0 \\ 0 & 1 \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} \mathbb{D}' \\ 1 \end{smallmatrix}\right) \subseteq \mathbb{Y},$$

where the subset is proper for $k > 1$. This assertion is equivalent, considering all the previous assumptions, to

$$\frac{a+b}{c+d} \leq \frac{2^{k-1}a + b}{2^{k-1}c + d} < \frac{2^k a - b}{2^k c - d} \leq \frac{2a - b}{2c - d}\ .$$

[11] We may multiply both sides by the two denominators without worrying about the direction of inequality as both denominators have equal sign. This comes from the fact that if $b_p = 0$, then $\gamma_p(\mathbb{D})$ is exclusively either in positive or negative part of $\mathbb{R}^*$.

We will prove the three inequalities separately. Remarked earlier, we may correctly assume that all denominators have equal signs. Thus:

$$\begin{aligned} \frac{a+b}{c+d} &\leq \frac{2^{k-1}a+b}{2^{k-1}c+d} \\ 0 &\leq (2^{k-1}-1)(ad-bc) \end{aligned}$$

and

$$\begin{aligned} \frac{2^{k-1}a+b}{2^{k-1}c+d} &< \frac{2^{k}a-b}{2^{k}c-d} \\ 0 &< (2^{k-1}+2^{k})(ad-bc) \end{aligned}$$

and

$$\begin{aligned} \frac{2^{k}a-b}{2^{k}c-d} &\leq \frac{2a-b}{2c-d} \\ 0 &\leq (2^{k}-2)(ad-bc), \end{aligned}$$

which all hold with respect to Eq. (5.6). The equalities happen for $k = 1$.

One may equally show that for $0 < k_1 < k_2$ it holds that $\frac{2^{k_1-1}a+b}{2^{k_1-1}c+d} < \frac{2^{k_2-1}a+b}{2^{k_2-1}c+d}$ and $\frac{2^{k_2}a-b}{2^{k_2}c-d} < \frac{2^{k_1}a-b}{2^{k_1}c-d}$. This proves that, for γ_{p+2} and later, the subset relation is proper and it thus completes the proof. □

Corollary 33. *Consider the assumptions of Theorem 32 and further assume $p > 1$ and $b_{p-1} = 0$. Then the theorem's assertion in Eq. (5.5) holds with $\mathbb{D} = (1, \infty]$ and $\mathbb{D}' = (\frac{1}{2}, \infty]$.*

Proof. Combine the proof of Theorem 32 and the result of Theorem 31. □

Similarly, we could follow to prove contractivity of different patterns within ERBCL expansions. Since this is bit a routine, we omit it here.

Example 34. *Let us show the contractivity for $x = 5$ and $(11101\text{–}10\text{–}1010)_{ERBCL}$ as one of its ERBCL representations equivalent to that of Example 26. The results are given in Tab. 5.3. Notice that the range of $\gamma_i(\mathbb{D}_i)$ contracts except for few cases when the range remains same. This result comes, however, from some constraints of $\mathbb{D}_i$ that further restricts domains implied by ERBCL definition. Take e.g. line for $i = 10$ of the table; while the definition implies that $x_{i+1} \in [\infty, -2) \cup (1, \infty]$ when $b_i = 0$, one can show that if $b_{i-3}b_{i-2}b_{i-1}b_i = 0\text{–}10$ then x_{i+1} is restricted to only $(1, \infty]$.*

□

5.3 Revision of Arithmetic

The principles of arithmetic do not change except the decision procedure. The decision conditions have to be updated appropriately to the new intervals associated with individual digits. The same change concerns inputs' range estimation; in this case, however, the algorithm needs to keep track of digit patterns that restrict the expected domain of input arguments (see e.g. Theorem 31).

An important aspect relied on for non-redundant continued logarithms was well-definedness of the computed (B)LFT. Due to the redundancy, it may happen that even for a well-defined BLFT an emission introduces a singularity so that the transformed BLFT is no longer well-defined. The importance of well-definedness was in simplifying estimation of the range of the BLFT. For a non-well-defined BLFT, such an estimation becomes more complex.

Table 5.3: Example of ERBCL contractivity.

i	b_i	γ_i	$\mathbb{D}_i$	$\gamma_i(\mathbb{D}_i)$
0	1	$\left(\begin{smallmatrix}2&0\\0&1\end{smallmatrix}\right)$	$[\infty,-1)\cup(\frac{1}{2},\infty]$	$[\infty,-2)\cup(1,\infty]$
1	1	$\left(\begin{smallmatrix}4&0\\0&1\end{smallmatrix}\right)$	$[\infty,-1)\cup(\frac{1}{2},\infty]$	$[\infty,-4)\cup(2,\infty]$
2	1	$\left(\begin{smallmatrix}8&0\\0&1\end{smallmatrix}\right)$	$[\infty,-1)\cup(\frac{1}{2},\infty]$	$[\infty,-8)\cup(4,\infty]$
3	0	$\left(\begin{smallmatrix}8&8\\1&0\end{smallmatrix}\right)$	$[\infty,-2)\cup(1,\infty]$	$(4,16)$
4	1	$\left(\begin{smallmatrix}8&4\\1&0\end{smallmatrix}\right)$	$[\infty,-1)\cup(\frac{1}{2},\infty]$	$(4,16)$
5	–	$\left(\begin{smallmatrix}8&-4\\1&0\end{smallmatrix}\right)$	$(1,\infty]$	$(4,8]$
6	1	$\left(\begin{smallmatrix}8&-2\\1&0\end{smallmatrix}\right)$	$(\frac{1}{2},\infty]$	$(4,8]$
7	0	$\left(\begin{smallmatrix}6&8\\1&1\end{smallmatrix}\right)$	$[\infty,-2)\cup(1,\infty]$	$(4,7)$
8	–	$\left(\begin{smallmatrix}6&-8\\1&-1\end{smallmatrix}\right)$	$(2,\infty]$	$(4,6]$
9	1	$\left(\begin{smallmatrix}12&-8\\2&-1\end{smallmatrix}\right)$	$(1,\infty]$	$(4,6]$
10	0	$\left(\begin{smallmatrix}4&12\\1&2\end{smallmatrix}\right)$	$(1,\infty]$	$[4,\frac{16}{3})$
11	1	$\left(\begin{smallmatrix}4&6\\1&1\end{smallmatrix}\right)$	$(\frac{1}{2},\infty]$	$[4,\frac{16}{3}]$
12	0	$\left(\begin{smallmatrix}10&4\\2&1\end{smallmatrix}\right)$	$[\infty,-2)\cup(1,\infty]$	$(\frac{14}{3},\frac{16}{3})$

5.4 Computability Issues

In Section 5.1 and section 5.2, we showed that redundant continued logarithms satisfy the contractivity. Interval openness could be proved similarly. We thus conjecture that continued logarithms satisfy the interior containment property, which claims their representation to be potentially useful for computable real arithmetic. Utilizing this potential depends eventually on implementation of basic arithmetic operators.

In case of continued logarithms, all such operators collide into the universal BLFT operator. Backed by the lazy evaluation scheme, the BLFT operator is computable. This conjecture might likely be proved formally, but we prefer an intuitive approach here. There are two basic facts to consider: contraction of the approximating interval when elements of the representation are processed left-to-right, and secondly the character of a BLFT as a continuous[12] and monotone[13] map in $\mathbb{R}^* \times \mathbb{R}^*$. Due to the latter fact, the range of a BLFT will be bounded on a sufficiently small, bounded domain $\mathbb{D}$, represented by the estimated range of input arguments. As $\mathbb{D}$ will contract—and it will because of the continued logarithms' contractivity—the BLFT's range will contract too so that it finally meets some decision condition. That some condition will ever be met is ensured by overlaps of the neighboring decision intervals.

Example 35. *We will demonstrate the computability of redundant continued logarithms on the example of $\phi(x,y) = xy$ and $x = y = \sqrt{2}$. We expect to use Alg. 2 with modifications discussed in Section 5.3.*

The progress of the algorithm is given in Tab. 5.4. The table is indexed by the triple (m,n,o) and provides actual state of the computed BLFT $\phi^{(m;n,o)}$, estimated range $\mathbb{D}_{x_n}$ and $\mathbb{D}_{y_o}$ of input arguments' continuations, and the range of the BLFT for these arguments. Finally, the table provides output elements e_m and a an approximation interval $\gamma_m(\mathbb{D}_{\phi_m})$ associated with the prefix of ϕ's expansion computed thus far.

While for non-redundant continued logarithms the example is uncomputable, redundant representation will successfully provide approximation of the expected result. We should also remark here on a difference between binary and canonical form. As we see from the table, in binary representation we obtained the expansion of $(10111\ldots)_{ERBCL}$, where the continuation would expand to an infinite stream of '1's. Extending this expansion the approximating interval $\gamma_m(\mathbb{D}_{\phi_m})$

[12] A BLFT is actually *piecewise* continuous. But, omitting undefined cases of $\frac{0}{0}$, we may let $\frac{axy+by+cx+d}{0} = \infty$ for an x and y such that $axy + by + cx + d \neq 0$.

[13] The monotonicity holds for sufficiently small connected subsets of the BLFT's domain.

would converge to $\{2\}$*. On the other hand, when using the canonical representation, we would get a single element of the result:* $(1, 1, 1)$*. No next term would ever be produced, unless* x *or* y *had differed from* $\sqrt{2}$*. That is, thanks to its finer granularity, the binary representation provides more information on the actual value of the computed quantity as the number of trailing '1's indicates how close to that quantity our computed result is.*

Table 5.4: A snapshot of variables and parameters taken at line 5 of Alg. 2 computed for ERBCL and $\phi = xy$, $x = y = \sqrt{2}$. To save space, we use $(b, -a)$ symbolic for $a, b > 0$ to denote $[\infty, -a) \cup (b, \infty]$.

m	n	o	$\phi^{(m;n,o)}$	e_m	$\mathbb{D}_{x_n}$	$\mathbb{D}_{y_o}$	$\phi^{(m;n,o)}(\mathbb{D}_{x_n} \times \mathbb{D}_{y_o})$	$\gamma_m(\mathbb{D}_{\phi_m})$
0	0	0	$\left(\begin{smallmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{smallmatrix}\right)$	$\emptyset$	$\mathbb{R}^*$	$\mathbb{R}^*$	$\mathbb{R}^*$	$\mathbb{R}^*$
0	1	1	$\left(\begin{smallmatrix} 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \end{smallmatrix}\right)$	$\emptyset$	$(1, -2)$	$(1, -2)$	$(\frac{1}{4}, 4)$	$\mathbb{R}^*$
0	3	3	$\left(\begin{smallmatrix} 9 & 6 & 6 & 4 \\ 4 & 4 & 4 & 4 \end{smallmatrix}\right)$	1	$(\frac{1}{2}, -1)$	$(\frac{1}{2}, -1)$	$(\frac{25}{16}, 4)$	$(1, -2)$
1	3	3	$\left(\begin{smallmatrix} 9 & 6 & 6 & 4 \\ 8 & 8 & 8 & 8 \end{smallmatrix}\right)$	0	$(\frac{1}{2}, -1)$	$(\frac{1}{2}, -1)$	$(\frac{25}{32}, 2)$	$(1, 4)$
2	3	3	$\left(\begin{smallmatrix} 8 & 8 & 8 & 8 \\ 1 & -2 & -2 & -4 \end{smallmatrix}\right)$	1	$(\frac{1}{2}, -1)$	$(\frac{1}{2}, -1)$	$(1, -\frac{32}{7})$	$(1, 4)$
3	5	4	$\left(\begin{smallmatrix} 8 & 2 & 4 & 1 \\ 2 & -1 & -2 & -1 \end{smallmatrix}\right)$	$\emptyset$	$(\frac{1}{2}, -1)$	$(\frac{1}{2}, -1)$	$(\frac{3}{4}, -3)$	$(1, 4)$
3	5	5	$\left(\begin{smallmatrix} 16 & 4 & 4 & 1 \\ 4 & -2 & -2 & -1 \end{smallmatrix}\right)$	1	$(\frac{1}{2}, -1)$	$(\frac{1}{2}, -1)$	$(\frac{9}{7}, -\frac{17}{4})$	$(\frac{3}{2}, 3)$
4	5	5	$\left(\begin{smallmatrix} 16 & 4 & 4 & 1 \\ 8 & -4 & -4 & -2 \end{smallmatrix}\right)$	$\emptyset$	$(\frac{1}{2}, -1)$	$(\frac{1}{2}, -1)$	$(\frac{9}{14}, -\frac{17}{8})$	$(\frac{3}{2}, 3)$
4	6	5	$\left(\begin{smallmatrix} 20 & 5 & 16 & 4 \\ 4 & -6 & 8 & -4 \end{smallmatrix}\right)$	$\emptyset$	$(1, -2)$	$(\frac{1}{2}, -1)$	$(\frac{91}{52}, -\frac{5}{6})$	$(\frac{3}{2}, 3)$
4	6	6	$\left(\begin{smallmatrix} 25 & 20 & 20 & 16 \\ -2 & 4 & 4 & 8 \end{smallmatrix}\right)$	1	$(1, -2)$	$(1, -2)$	$(\frac{81}{14}, -\frac{9}{4})$	$(\frac{7}{4}, \frac{5}{2})$
...	...	...	...	...	...	...	...	...

□

5.5 Speculative Redundancy

There is a form of a computable representation that is an alternative to fully redundant ERBCL. We call it the *speculatively redundant continued logarithm* as it combines redundancy introduced by ERBCL with a principle of speculation. This representation behaves, as often as possible, like non-redundant EBCL, but in cases that are possibly undecidable it speculatively applies redundancy. The amount of speculation can vary. Usually, the decision algorithm tries to chose from the "non-redundant" decision conditions and, if not possible, it tries if some "redundant" decision case applies.

The speculatively redundant representation has advantages coming from both redundant and non-redundant representations. That is, it explicitly differentiate between digits produced non-speculatively and those generated speculatively—by doing so, the non-speculative digits have associated tighter domain and thus provide tighter bounds on the represented quantity. On the other hand, the redundancy, even if applied speculatively, provides a means for solving the computability issue.

This speculative approach also simplifies some aspects of practical implementation, namely those discussed in Section 5.3. Some further details may be found in [Brabec, 2006], but the full overview of the idea of the speculative redundancy is to be published separately.

6 Conclusion

In this text we presented an overview of continued logarithms from perspective of computer arithmetic. We introduced some fundamental properties and principles of arithmetic operators that give this representation a potential of computability. We exploited this potential by introducing a redundant variant of the representation and proved its possession of the interior containment property as a necessary condition for computable real arithmetic. We tried to

illustrate all important aspects on examples that complements the theory and provide additional insight.

So far, the text provides the basic theory and understanding of continued logarithms as a tool for exact real arithmetic. Yet there are topics missing in this text that should be added in the future[14]. Among those it is mainly an analysis similar to that of Heckmann [Heckmann, 1999] that would provide some clues on complexity issues of the continued logarithm arithmetic. There are also topics of practical interest: notes on the computer implementation of the discussed principles, advanced arithmetic for elementary functions, etc.

7 Bibliography

[Boehm et al., 1986] Boehm, H. J., Cartwright, R., Riggle, M., and O'Donnell, M. J. (1986). *Exact Real Arithmetic: A Case Study in Higher Order Programming.* In *ACM Symposium on Lisp and Functional Programming* (pp. 162–173).: ACM Press.

[Brabec, 2006] Brabec, T. (2006). *On Exact Real Hardware with Specialization to Continued Methods.* PhD Proposal, Dept. of CS&E, FEE, Czech Technical University. http://service.felk.cvut.cz/anc/brabect1/pub/phdprop06.pdf.

[Brabec, 2008] Brabec, T. (2008). Redundant Cont. Log. Representation: Proof of Contractivity. unpublished paper.

[Edalat & Heckmann, 2002] Edalat, A. and Heckmann, R. (2002). *Computing with Real Numbers: I. LFT Approach to Real Computation, II. Domain-theoretic Model of Computational Geometry.* In *Applied Semantics, International Summer School, APPSEM 2000, Caminha, Portugal, September 9-15, 2000, Advanced Lectures* (pp. 193–267). London, UK: Springer-Verlag.

[Gosper, 1972] Gosper, R. W. (1972). *Item 101 in Hakmem.* AIM239, MIT, (pp. 37–44).

[Gosper, 1978] Gosper, R. W. (1978). Continued Fraction Arithmetic. Unpublished Draft Paper.

[Gowland & Lester, 2000] Gowland, P. and Lester, D. (2000). *A Survey of Exact Arithmetic Implementations.* In J. Blanck, V. Brattka, & P. Hertling (Eds.), *Computability and Complexity in Analysis CCA 2000*, volume 2064 of *Lecture Notes in Computer Science* (pp. 30–47).: Springer.

[Heckmann, 1999] Heckmann, R. (1999). *How Many Argument Digits are Needed to Produce n Result Digits?* Electr. Notes Theor. Comput. Sci., 24, 13–33.

[Khinchin, 1964] Khinchin, A. (1964). *Continued Fractions, 3rd ed.* The University of Chicago Press. Translated from Russian by P. Wynn, P. Noordhoff Ltd., Grooningen, 1963. Also by H. Eagle, The University of Chicago Press, 1964.

[Kornerup & Matula, 1990] Kornerup, P. and Matula, D. W. (1990). *An Algorithm for Redundant Binary Bit-Pipelined Rational Arithmetic.* IEEE Transactions on Computers, C-39(8), 1106–1115.

[Nielsen, 1993] Nielsen, F. (1993). *Algorithms on Continued and Multi-continued fractions.* Master's thesis, Ecole Normale Superieure Lyon, Lyon, France.

[14] Since this text will be available on-line, we will keep it updated as new information will be added.

[Niqui, 2007] Niqui, M. (2007). *Exact Arithmetic on the Stern–Brocot Tree.* Journal of Discrete Algorithms, 5(2), 356–379.

[Potts, 1998] Potts, P. J. (1998). *Exact Real Arithmetic using Möbius Transformations.* Phd thesis, University of London, Imperial College.

[Vuillemin, 1990] Vuillemin, J. (1990). *Exact Real Computer Arithmetic with Continued Fractions.* IEEE Transactions on Computers, C-39(8), 1087–1105.

[Wiedmer, 1980] Wiedmer, E. (1980). *Computing with Infinite Objects.* Theoretical Computer Science, 10, 133–155.